

高性能计算导论

第2讲：并行编程模型

翟季冬
计算机系

目录

- 并行计算概述
- 并行计算硬件模型
- 并行编程模型
- 并行计算性能指标

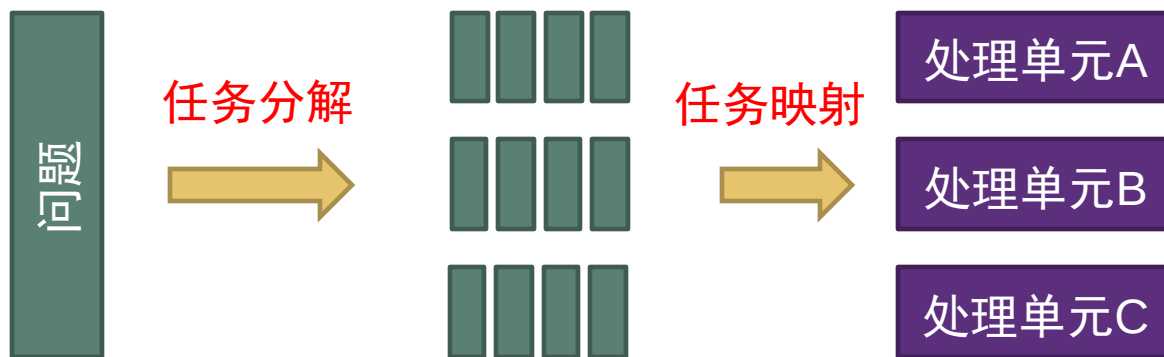
并行计算概述

什么是并行计算？

- 并行计算：使用多个处理单元协同解决问题
- 串行计算：



- 并行计算：



为什么需要并行计算？

- 追求高性能（节省时间）
 - 实时场景对**计算时间**具有严格要求
 - 例如：自动驾驶
 - 节省时间往往意味着**成本降低**
 - 例如：云计算
 - 节省时间允许运行**更多任务**或进行**更多调优**
 - 例如：深度学习训练



	Dual Xeon CPU Server	DGX-1 Server (8 V100 GPUs)
Flop/s	3T	170T
内存带宽	76 GB/s	768 GB/s
AlexNet训练时间	150小时	2小时
若2小时训练完成	>250节点	1节点

为什么需要并行计算？

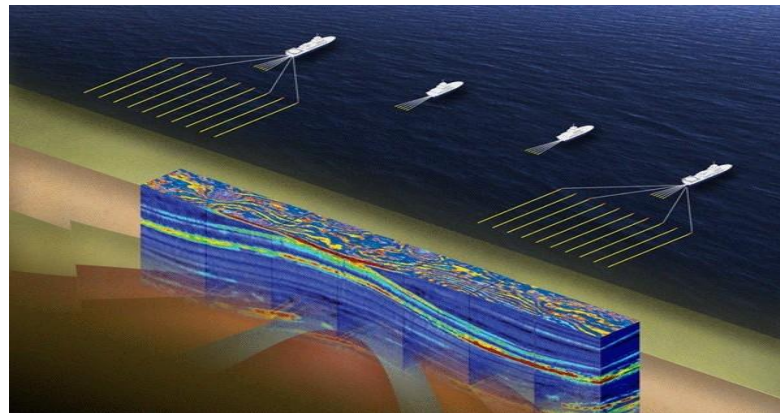
■ 解决更大问题



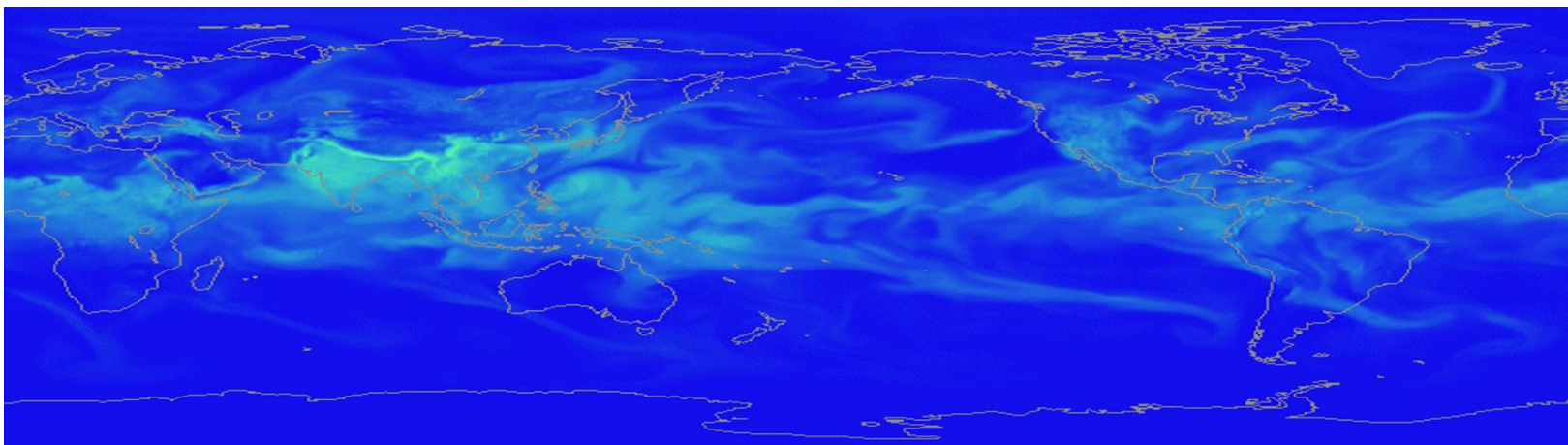
弹道计算



核爆模拟



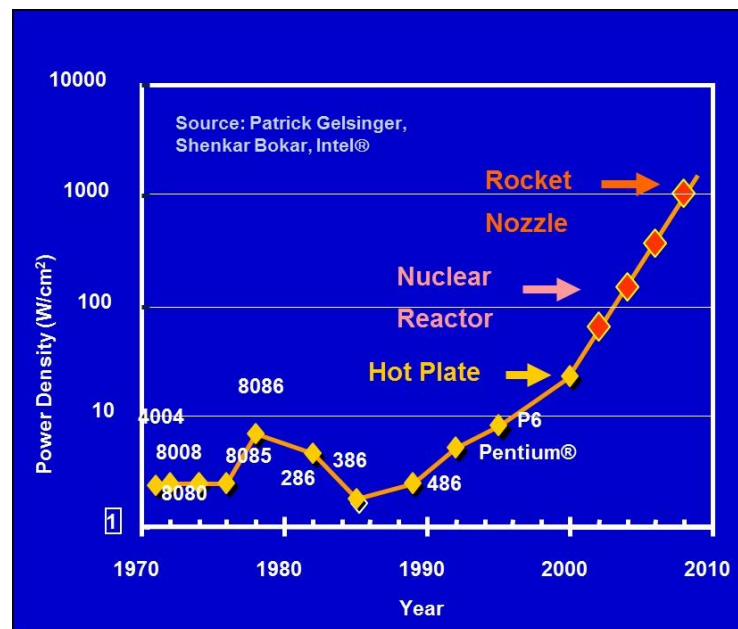
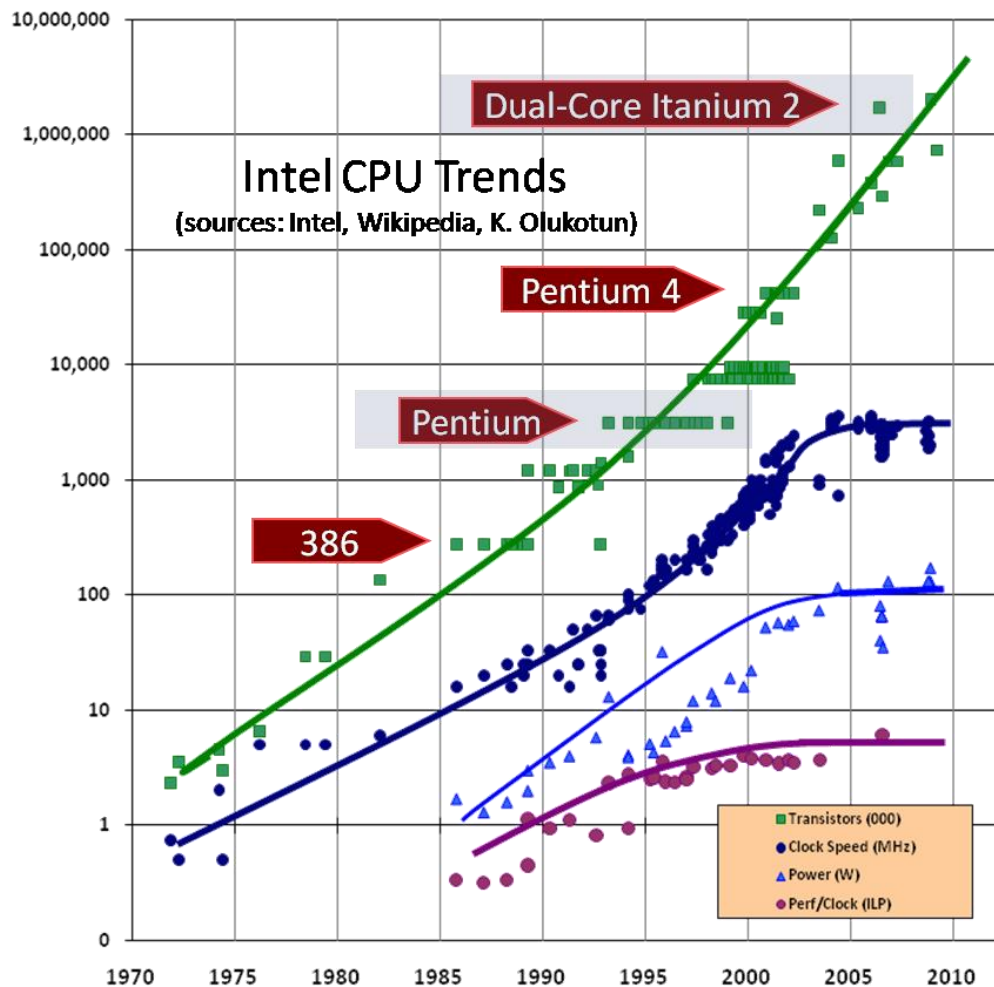
地质勘探



天气预报

为什么需要并行计算？

多核的趋势



主频受**功耗限制**难以继续提升

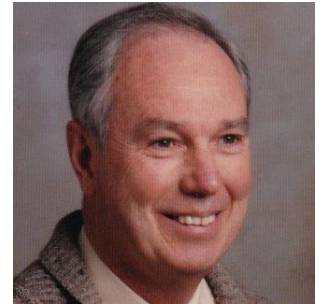
核趋向**简单**、**核数增加**成为趋势

并行计算硬件模型

并行计算硬件模型

■ Flynn分类法（佛林分类法）

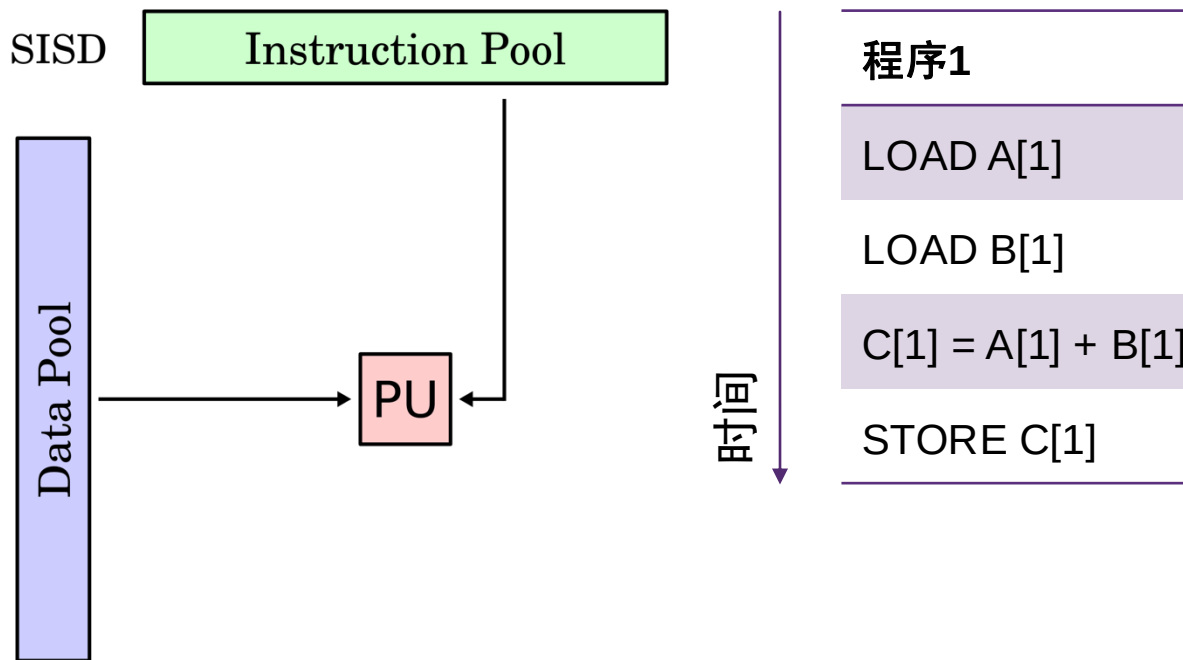
- Michael J. Flynn于1966年提出，一直以来被用于指导现代处理器设计
- 从处理单元角度分类，基于两个维度：**指令流**与**数据流**（instruction stream and data stream）



	数据流-Data Stream	
指令流-Instruction Stream	<u>SISD</u> Single Instruction stream Single Data stream	<u>SIMD</u> Single Instruction stream Multiple Data streams
	<u>MISD</u> Multiple Instruction streams Single Data stream	<u>MIMD</u> Multiple Instruction streams Multiple Data streams

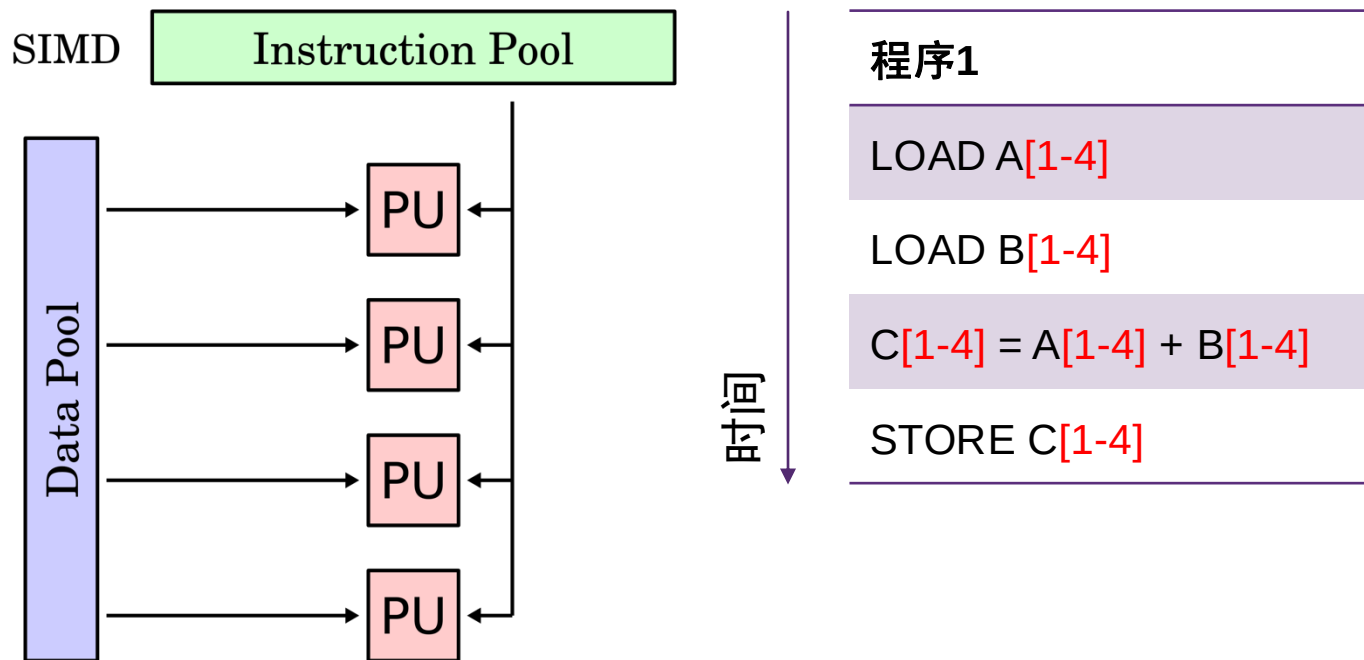
Flynn分类法：SISD

- 单指令流、单数据流
 - 单指令流：处理单元只运行一个指令流
 - 单数据流：处理单元只操纵一个数据流
 - 例子：单核处理器



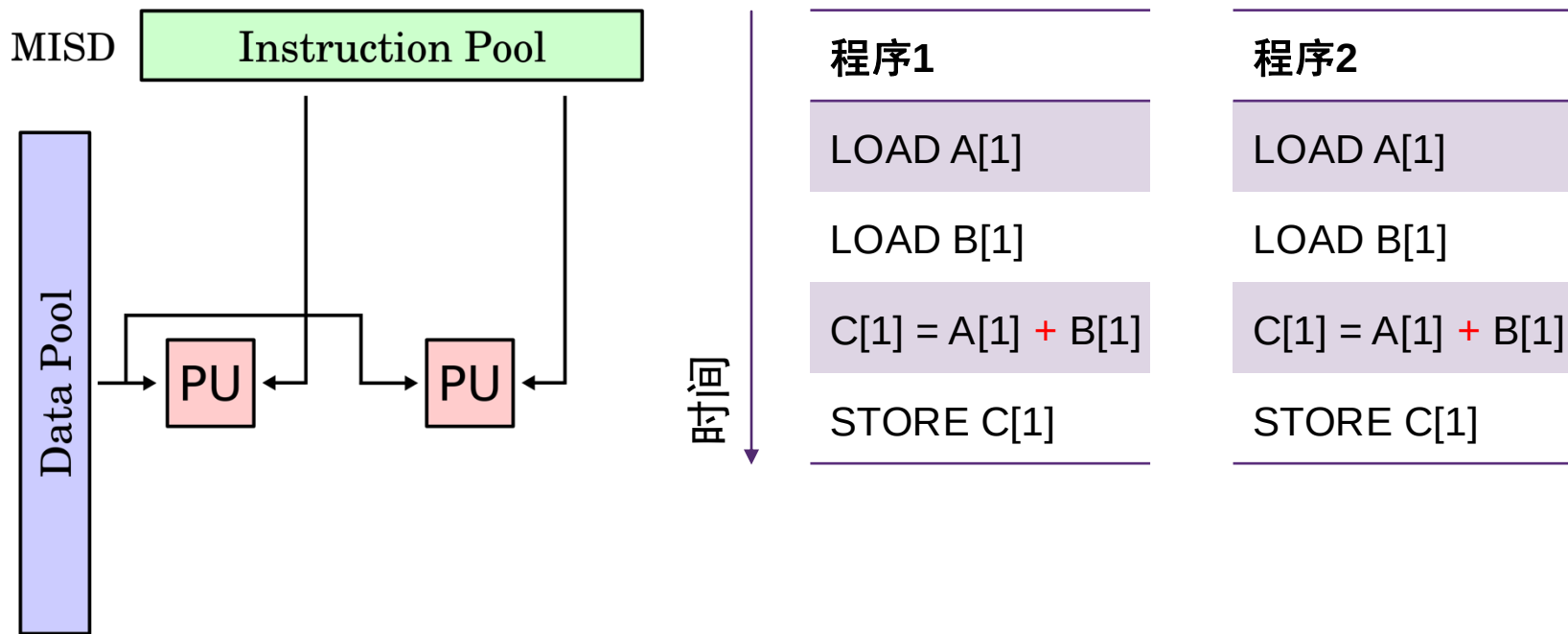
Flynn分类法：SIMD

- 单指令流、多数据流
 - 单指令流：所有处理单元运行**相同的指令流**
 - 多数据流：不同处理单元操纵**不同的数据流**
 - 例子：CPU中的向量处理单元(SIMD)



Flynn分类法：MISD

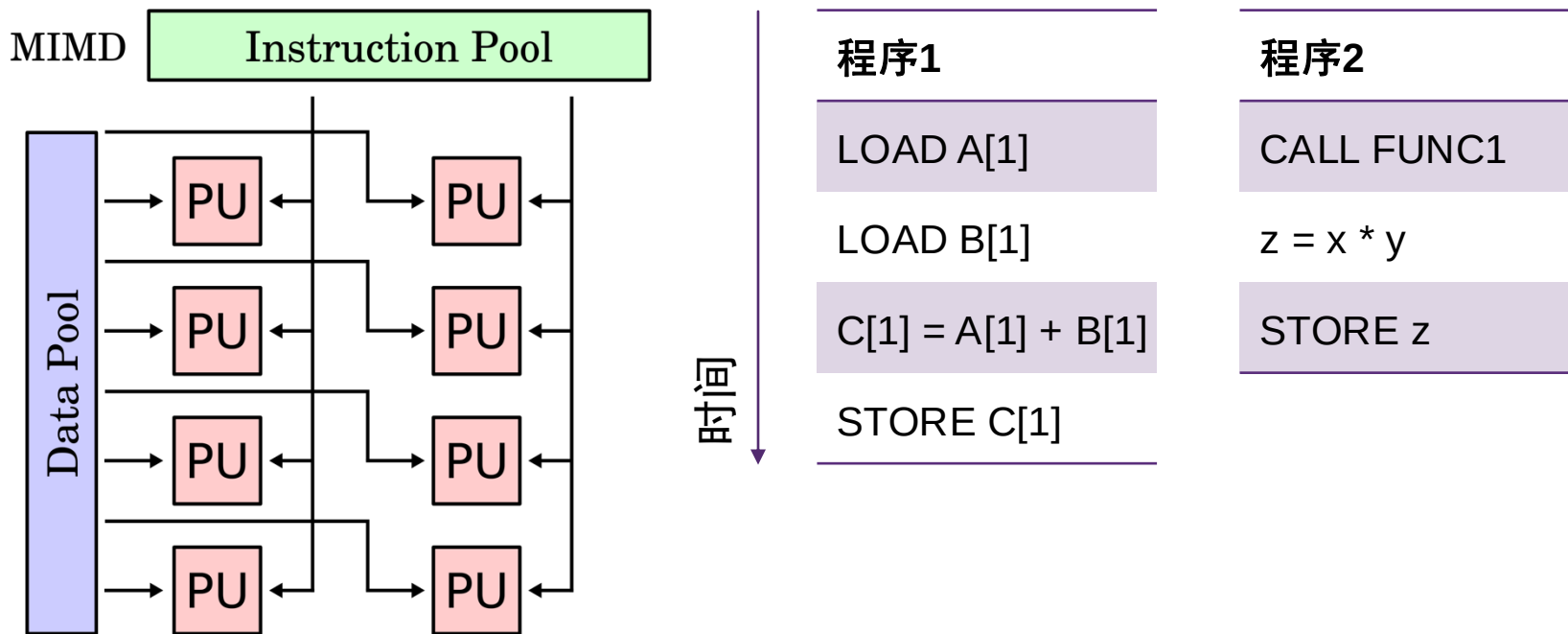
- 多指令流、单数据流
 - 多指令流：不同处理单元运行不同的指令流
 - 单数据流：所有处理单元操纵相同的数据流
 - 例子：仅1971年CMU做过类似试验；可被用于容错计算



Flynn分类法：MIMD

- 多指令流、多数据流

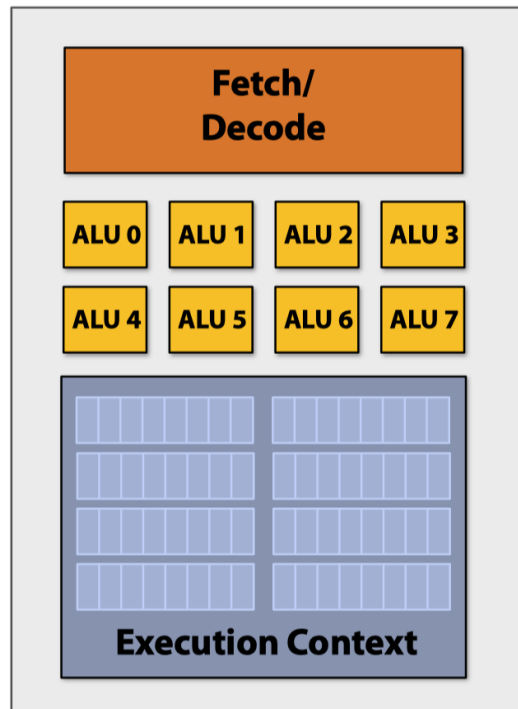
- 多指令流：不同处理单元（可能）运行不同的指令流
- 多数据流：不同处理单元（可能）操纵不同的数据流
- 例子：现代计算机、多核处理器



SIMD示例

SIMD

- 通常实现于处理器内
- 单个控制器控制多个处理单元
- 控制器发射指令，所有处理单元在不同的数据上执行该指令



SIMD示例-1

- 示例一：Intel CPU上的向量扩展指令
 - **SSE**（Streaming SIMD Extensions）：128位宽
 - XMM 0-15
 - **AVX**（Advanced Vector Extensions）：256位宽
 - YMM 0-15
 - **AVX-512**：512位宽
 - ZMM 0-31

SSE和AVX数据类型

SSE Data Types (16 XMM Registers)

__m128	Float	Float	Float	Float	4x 32-bit float											
__m128d	Double		Double		2x 64-bit double											
__m128i	B	B	B	B	B	B	B	B	B	B	B	B	B	B	B	16x 8-bit byte
__m128i	short	short	short	short	short	short	short	short	short	8x 16-bit short						
__m128i	int	int	int	int	4x 32bit integer											
__m128i	long long		long long		2x 64bit long											
__m128i	doublequadword				1x 128-bit quad											

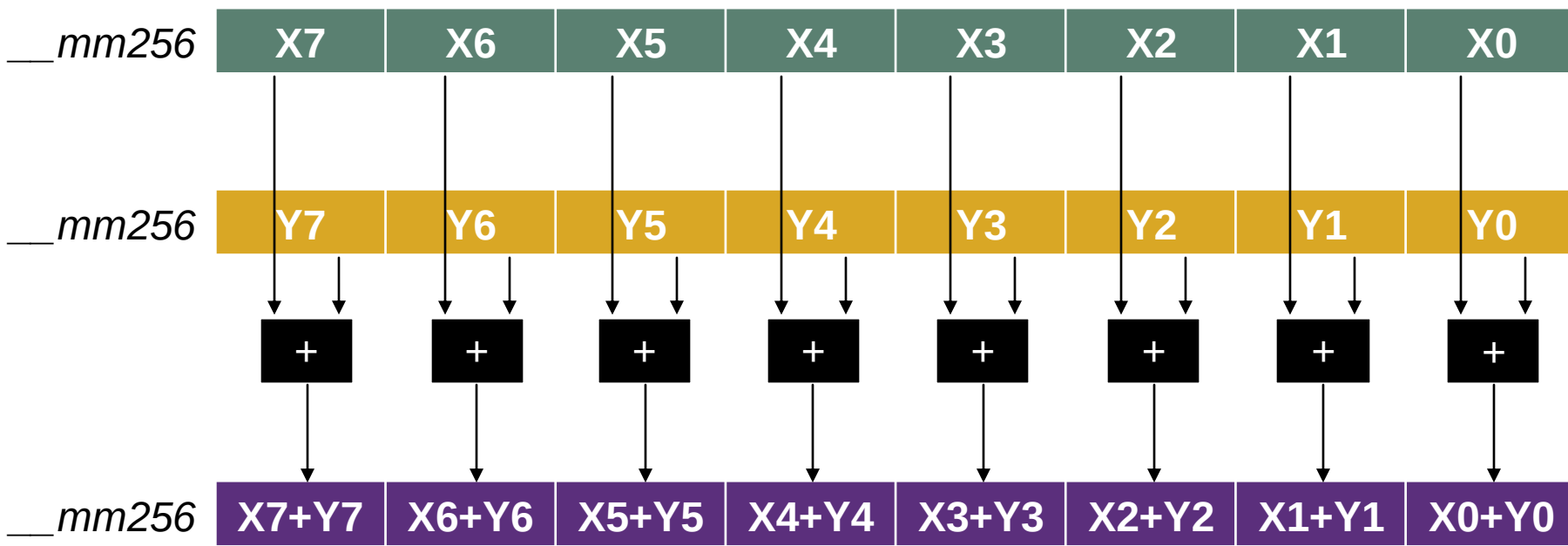
AVX Data Types (16 YMM Registers)

__mm256	Float	Float	Float	Float	Float	Float	Float	8x 32-bit float
__mm256d	Double		Double		Double		Double	4x 64-bit double
__mm256i	256-bit Integer registers. It behaves similarly to __m128i. Out of scope in AVX, useful on AVX2							

512	256 255	128 127	0
AVX-512	AVX	SSE	
ZMM0	YMM0	XMM0	
ZMM1	YMM1	XMM1	
ZMM2	YMM2	XMM2	
ZMM3	YMM3	XMM3	
ZMM4	YMM4	XMM4	
ZMM5	YMM5	XMM5	
ZMM6	YMM6	XMM6	
ZMM7	YMM7	XMM7	
ZMM8	YMM8	XMM8	
ZMM9	YMM9	XMM9	
ZMM10	YMM10	XMM10	
ZMM11	YMM11	XMM11	
ZMM12	YMM12	XMM12	
ZMM13	YMM13	XMM13	
ZMM14	YMM14	XMM14	
ZMM15	YMM15	XMM15	
ZMM16	YMM16	XMM16	
ZMM17	YMM17	XMM17	
ZMM18	YMM18	XMM18	
ZMM19	YMM19	XMM19	
ZMM20	YMM20	XMM20	
ZMM21	YMM21	XMM21	
ZMM22	YMM22	XMM22	
ZMM23	YMM23	XMM23	
ZMM24	YMM24	XMM24	
ZMM25	YMM25	XMM25	
ZMM26	YMM26	XMM26	
ZMM27	YMM27	XMM27	
ZMM28	YMM28	XMM28	
ZMM29	YMM29	XMM29	
ZMM30	YMM30	XMM30	
ZMM31	YMM31	XMM31	

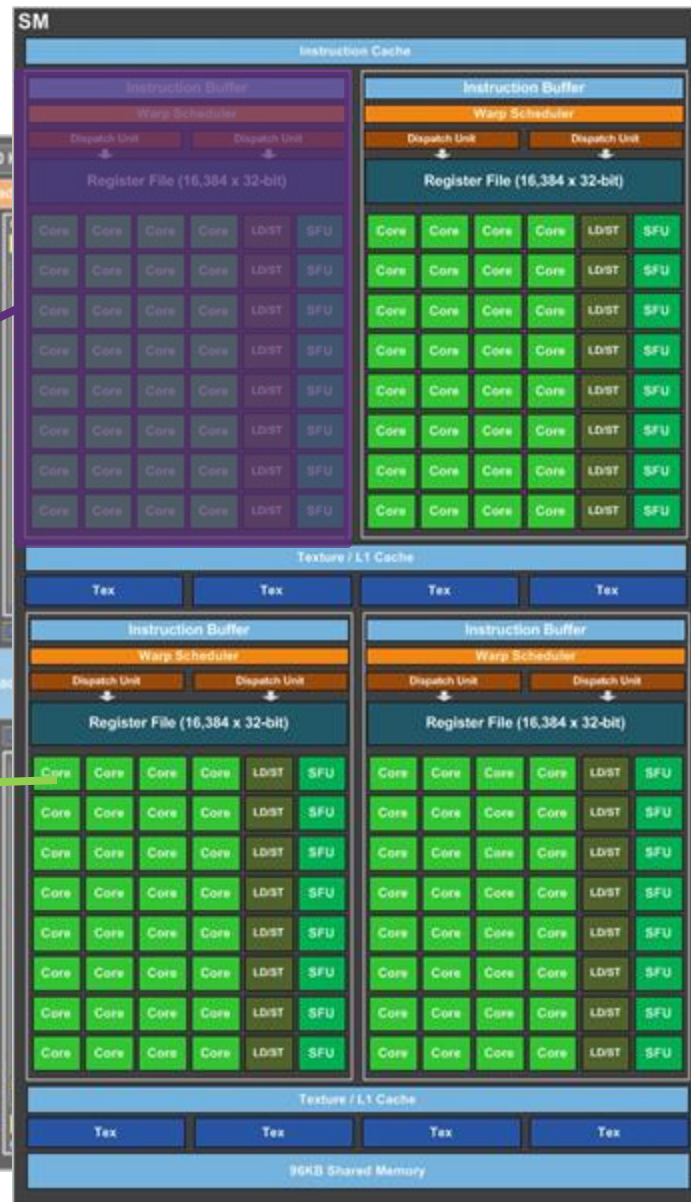
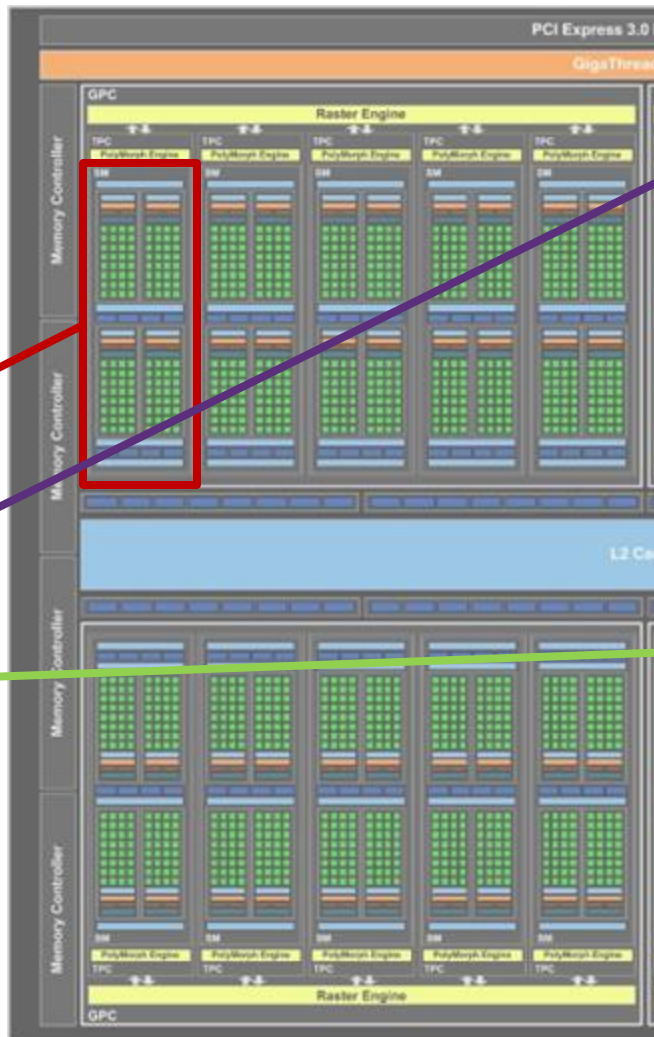
SIMD示例-1

- 示例一：Intel CPU上的向量扩展指令
 - 加法指令示例：AVX
 - `_mm256_add_epi32`（8个整数同时执行add）



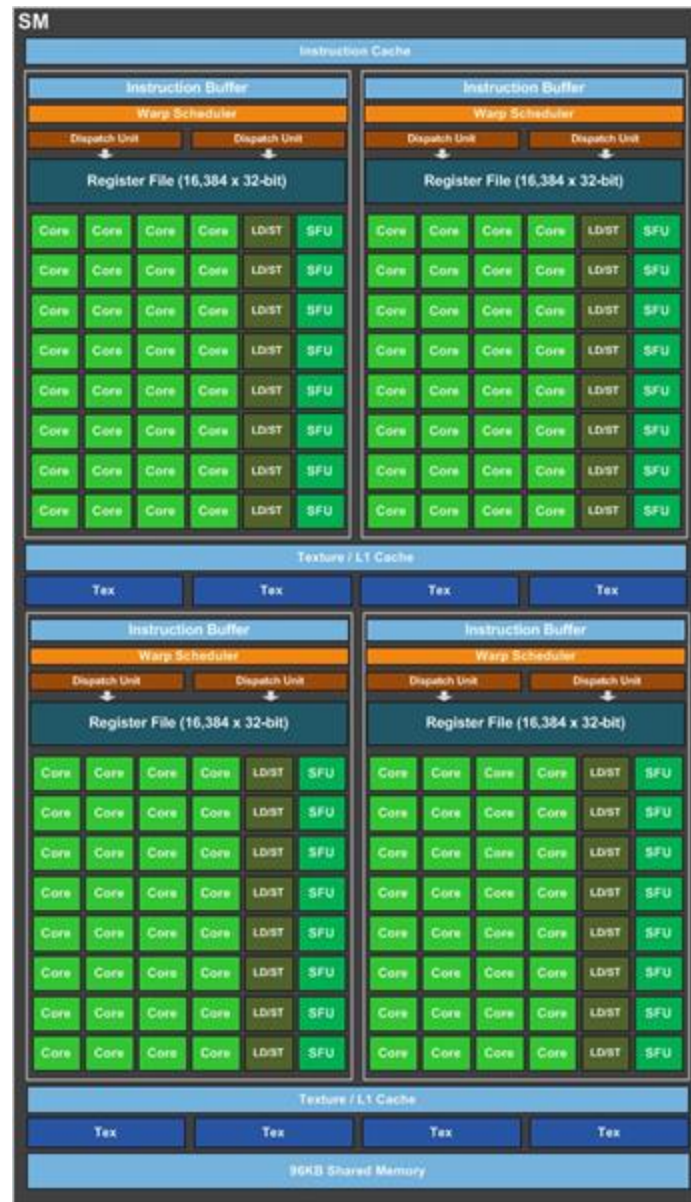
SIMD示例-2

- 示例二：
NVIDIA GPU
- GPU硬件层次
 - GPU
 - Streaming Multiprocessor (SM)
 - Warp
 - Core



SIMD示例-2

- 示例二：NVIDIA GPU
- Warp是指令执行的基本单元
 - 一条指令作用于一个Warp内的所有Core
 - Core操作在独立的数据上
- 1 Warp包含32 Cores，因而一个指令可操作于32个不同数据上
 - CPU的SIMD模式规定了操作数的总比特数
 - GPU的SIMD模式规定了操作数的个数
- NVIDIA将这种操作方式称为SIMT（Single Instruction Multiple Threads）



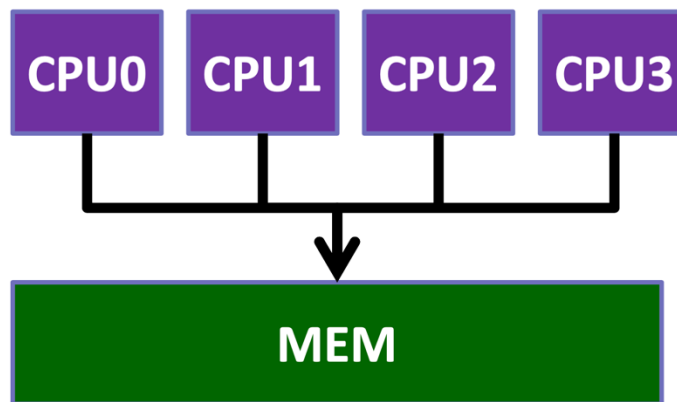
内存视角硬件分类

内存视角分类

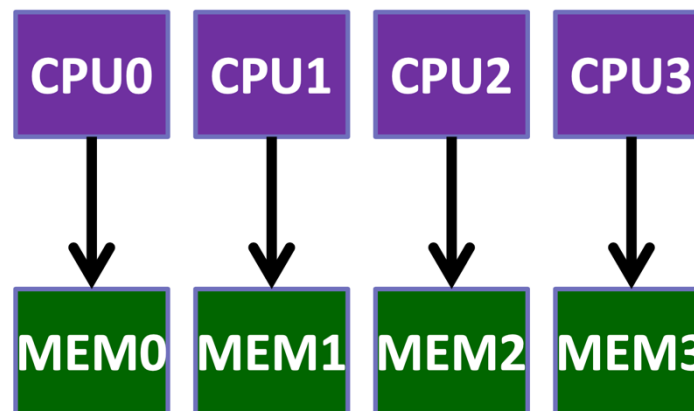
- 并行计算分类：
 - Flynn分类法从处理单元分类
 - 也可以从内存视角角度分类

常见的内存架构有：

- **共享内存** (Shared Memory)
- **分布式内存** (Distributed Memory)



共享内存



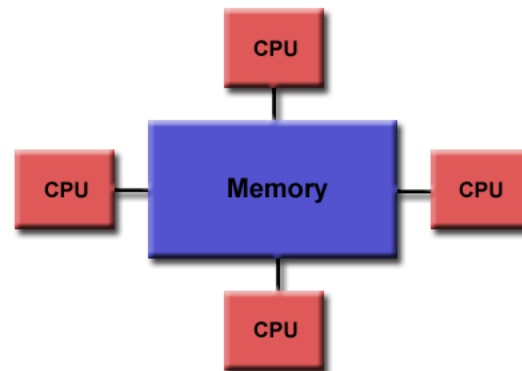
分布式内存

共享内存（Shared Memory）

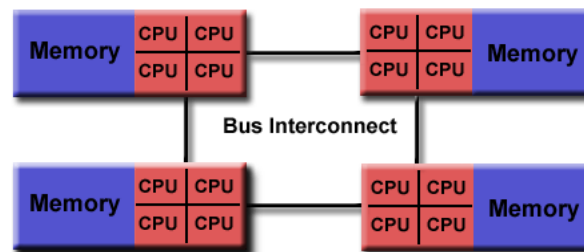
- 共享内存：
 - 所有处理单元与共享内存相连，处于同一个地址空间下
 - 任何处理单元可通过地址直接访问任意内存位置
 - 分类：UMA、NUMA

共享内存架构

- **均匀内存访问架构** (Unified Memory Access, **UMA**)
 - 每个处理器**访存性能相同**
 - 很难扩展到更大规模 (~8)
- **非均匀内存访问架构** (Non-Uniform Memory Access, **NUMA**)
 - 通常由2个以上计算单元组成
 - 每个单元拥有自己**局部内存**
 - 跨单元的**远程内存访问**代价较高



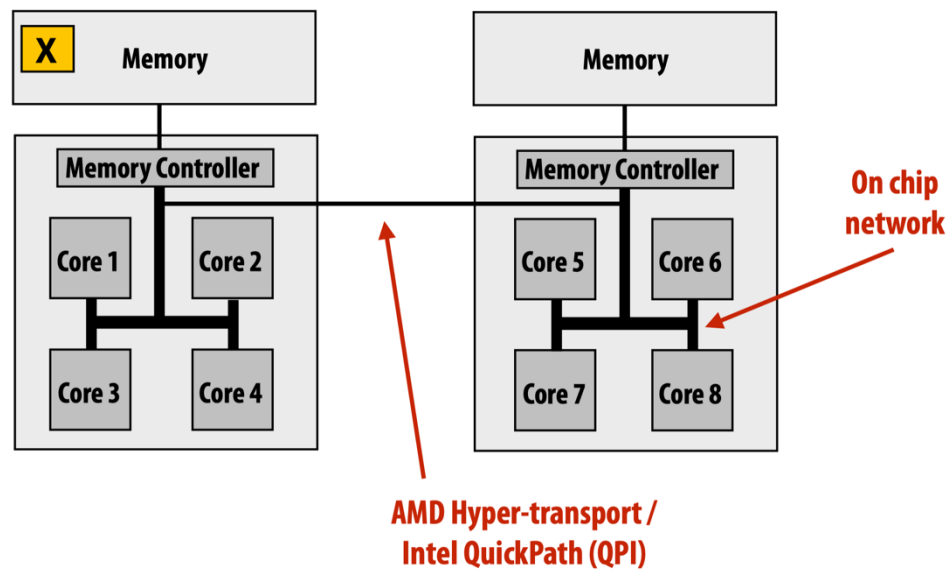
均匀内存访问架构-UMA



非均匀内存访问架构-NUMA

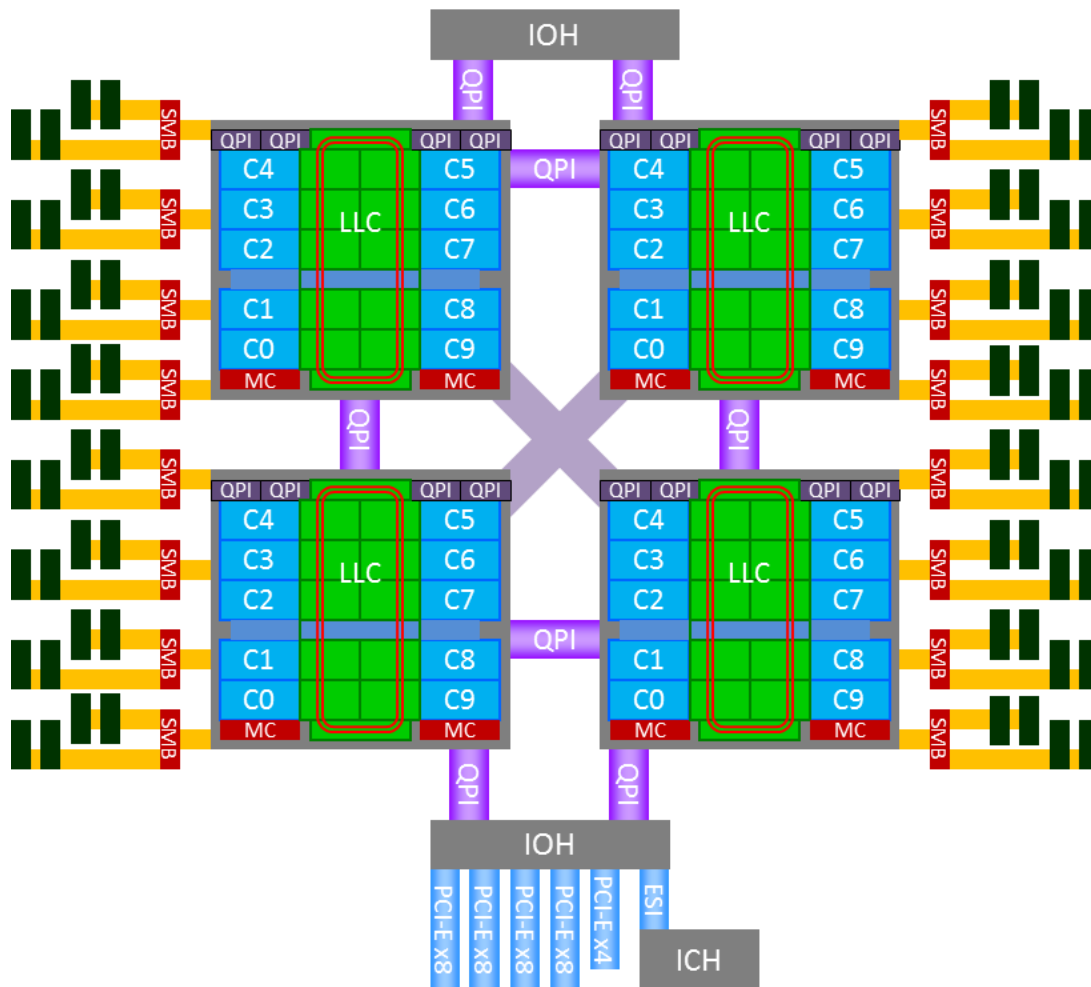
共享内存服务器-1

- 例：两路CPU服务器（2-way CPU）
 - NUMA架构



共享内存服务器-2

- 英特尔服务器：
 - 4个英特尔CPU
 - QPI技术互连
 - 每个CPU包括10个核
 - NUMA架构

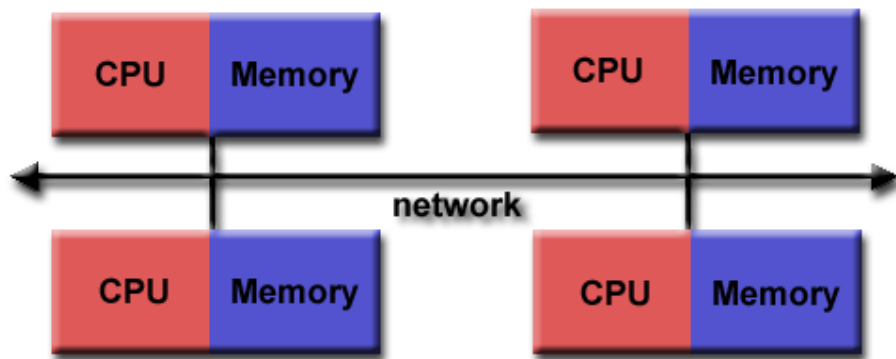


分布式内存（Distributed Memory）

- 分布式内存：

每个处理单元拥有自己的内存，不同内存的地址空间独立
故一个处理单元不可通过地址直接访问其它处理单元的内存

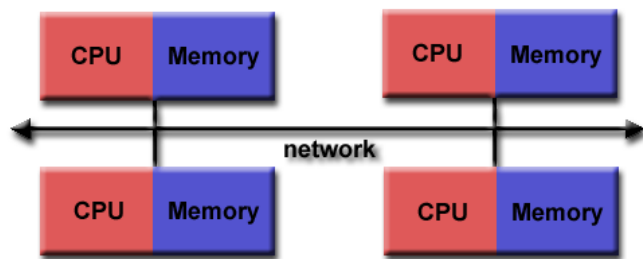
- 通过显式通信进行数据传输
- 例子：集群服务器



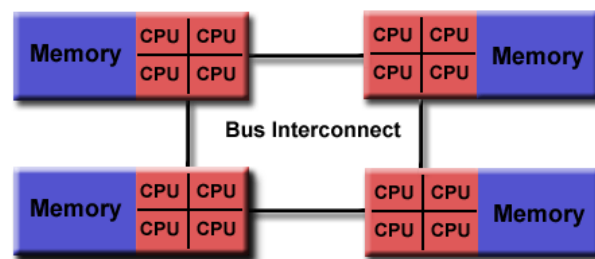
分布式内存架构

分布式内存 vs. 非均匀内存

- 分布式内存 vs. 非均匀内存（NUMA）
 - 共同点：
 - 内存的逻辑存在形式均为分布式
 - 不同点：
 - 分布式内存处于不同的内存地址空间下
不可通过地址直接访问
 - 非均匀内存处于统一的内存地址空间下
可以通过地址直接访问任意内存位置



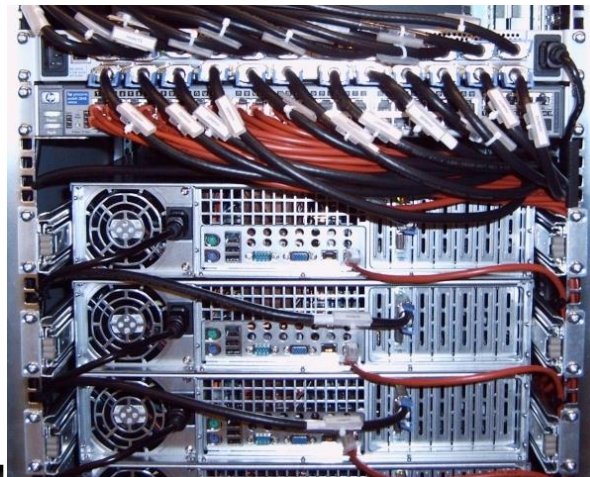
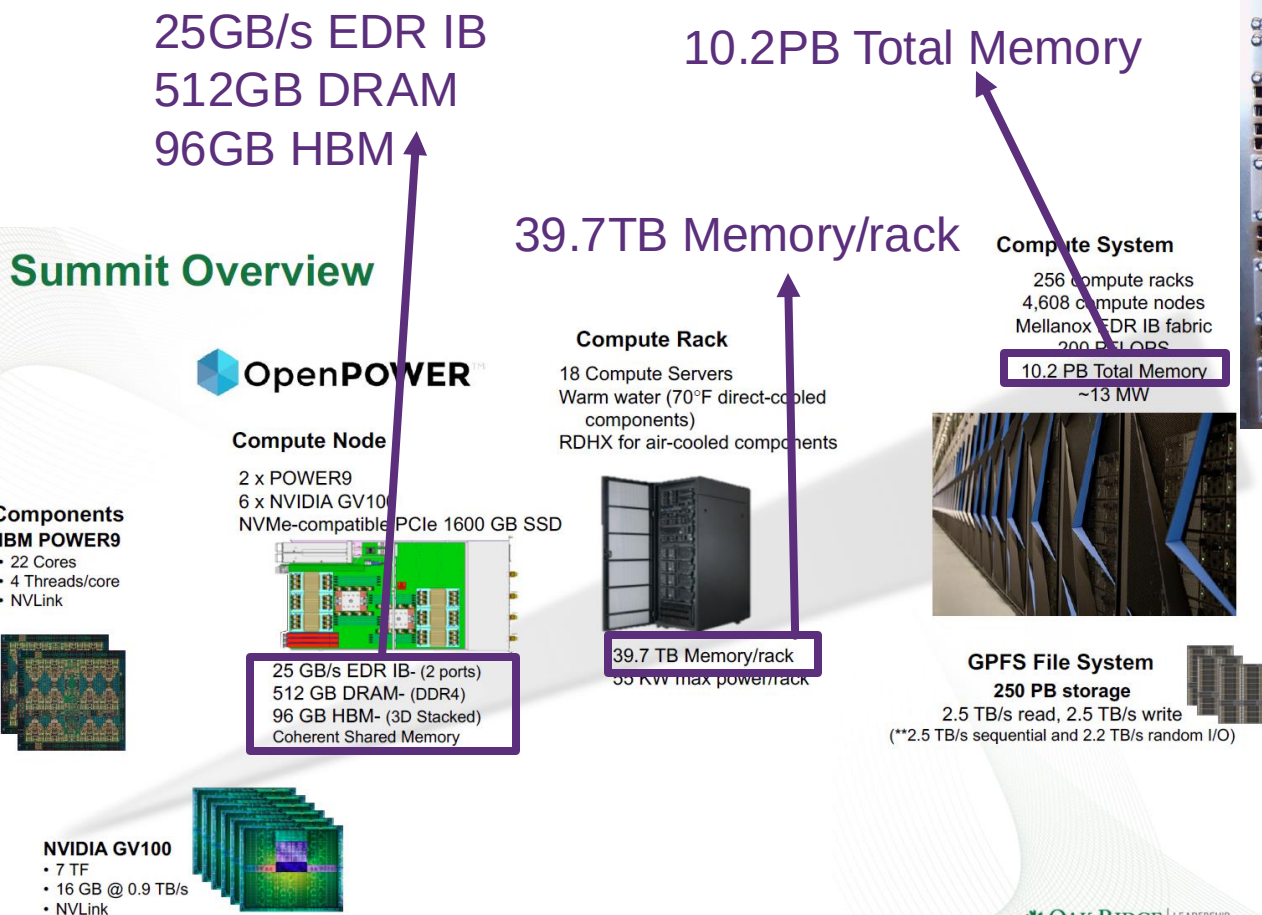
分布式内存架构



非均匀内存访问架构

分布式内存（Distributed Memory）

- 常见系统：由多台服务器组成的计算集群（数据中心、超算中心）
 - 处理单元：服务器

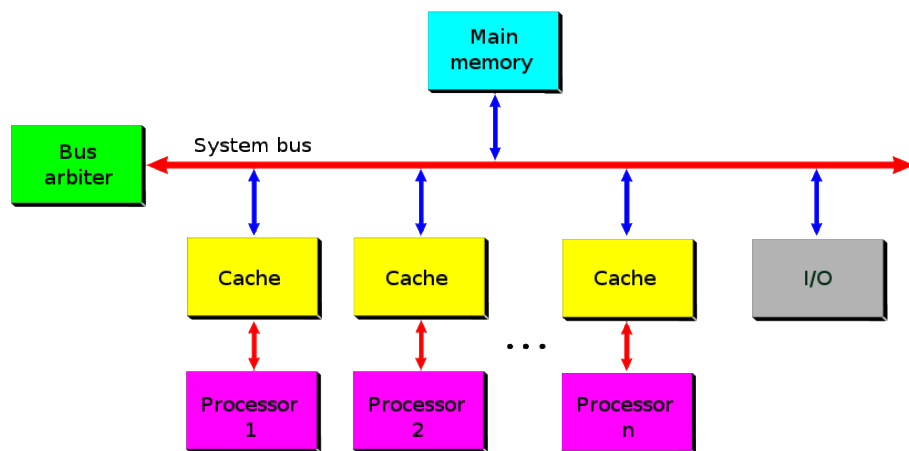


采用Infiniband互连的
集群服务器

处理器视角硬件分类

对称多处理器 - SMP

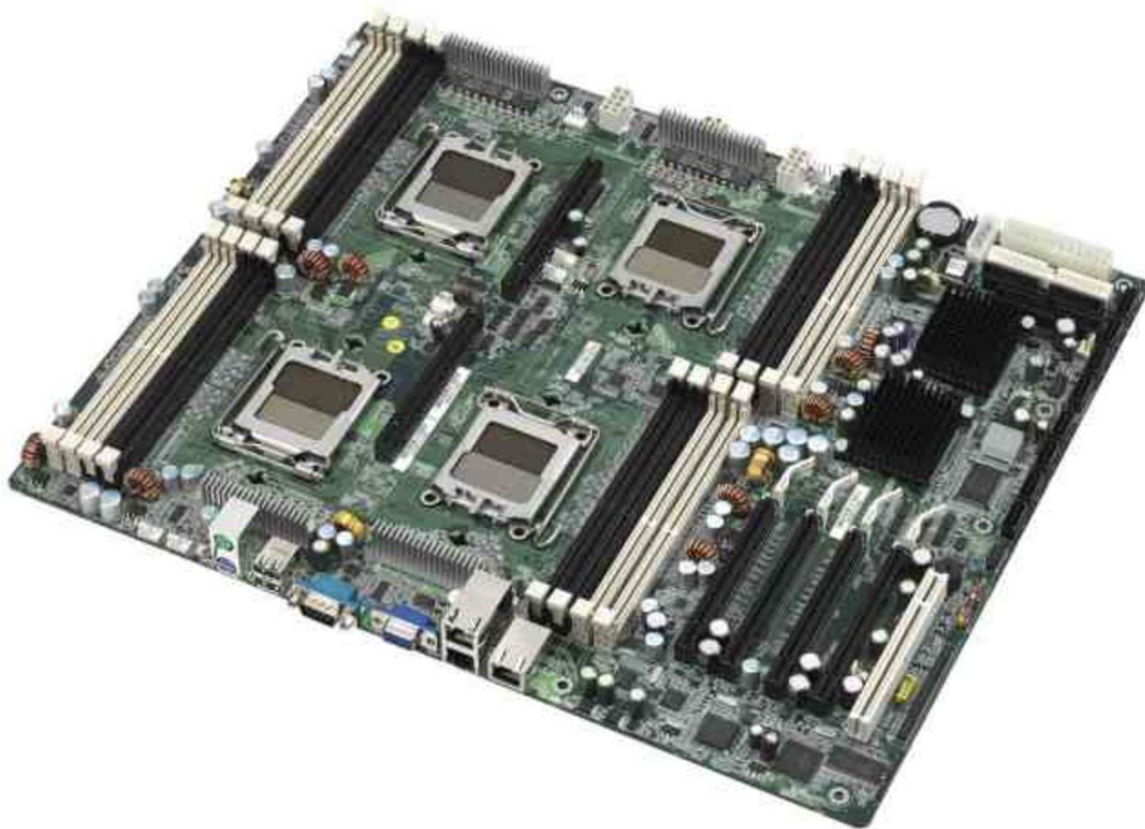
- **对称多处理器**（Symmetric Multiprocessor, SMP）
 - SMP：包括两个或多个相同处理器，连到一个共享内存
 - SMP 包括紧耦合**多个同构处理器**
 - 处理器是**对称的**
 - 每个处理器可以**同时处理不同任务**



对称多处理器

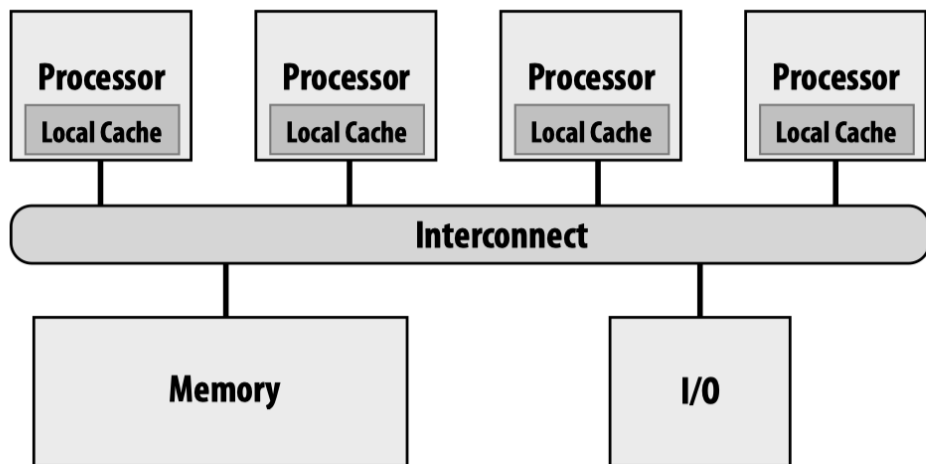
https://en.wikipedia.org/wiki/Symmetric_multiprocessing

SMP服务器主板

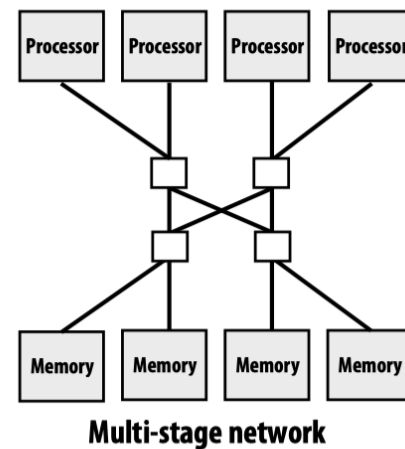
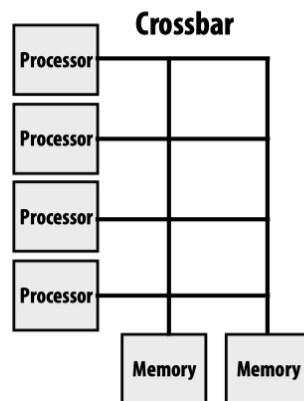
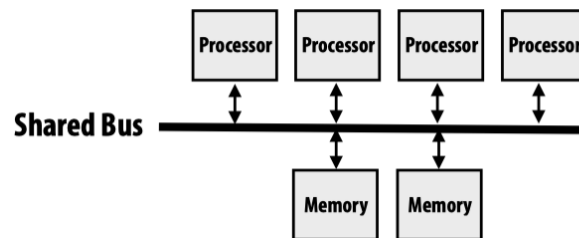


典型SMP互连

- SMP中典型处理器互连:



Interconnect examples

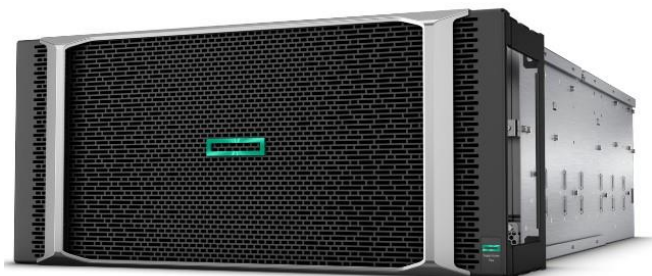


例子：HP Superdome服务器

- HP Superdome Flex服务器
 - 采用英特尔Skylake至强处理器
 - 最大支持32个处理器
 - 最大内存64TB
 - 采用NUMALink8互连技术
 - 处理器之间采用all-to-all全互连
 - 意味跨节点内存访问延迟相同
 - NUMA架构



不同配置支持最大内存



服务器前端



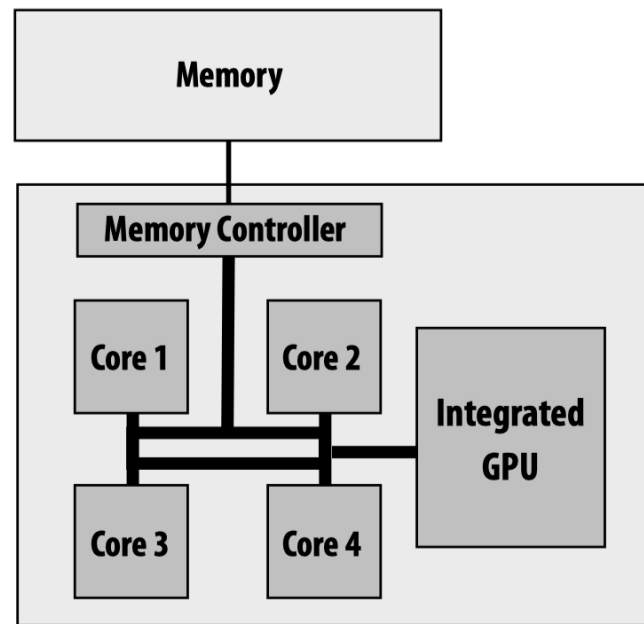
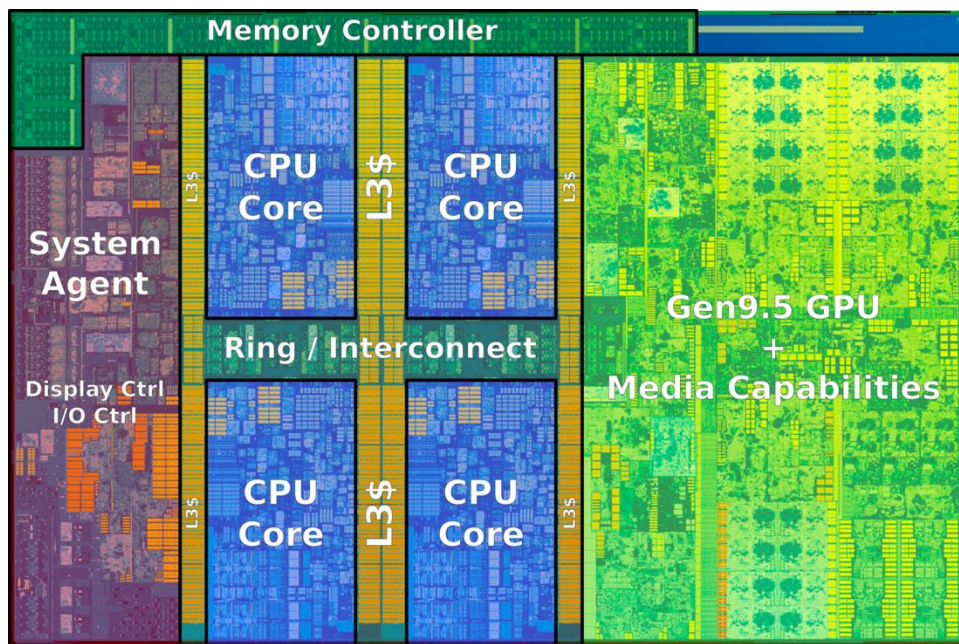
服务器后端

单芯片多处理器 - CMP

- **单芯片多处理器**（Chip Multi Processor - CMP）
 - 核心思想是将并行处理器中SMP技术应用到单个处理器内
 - 也叫**多核处理器**
- 在架构上和SMP类似，只是每个处理器内部又集成了大量独立处理器核心
 - 处理器核心之间可以有各种互连方式
- **常见硬件：多核服务器**

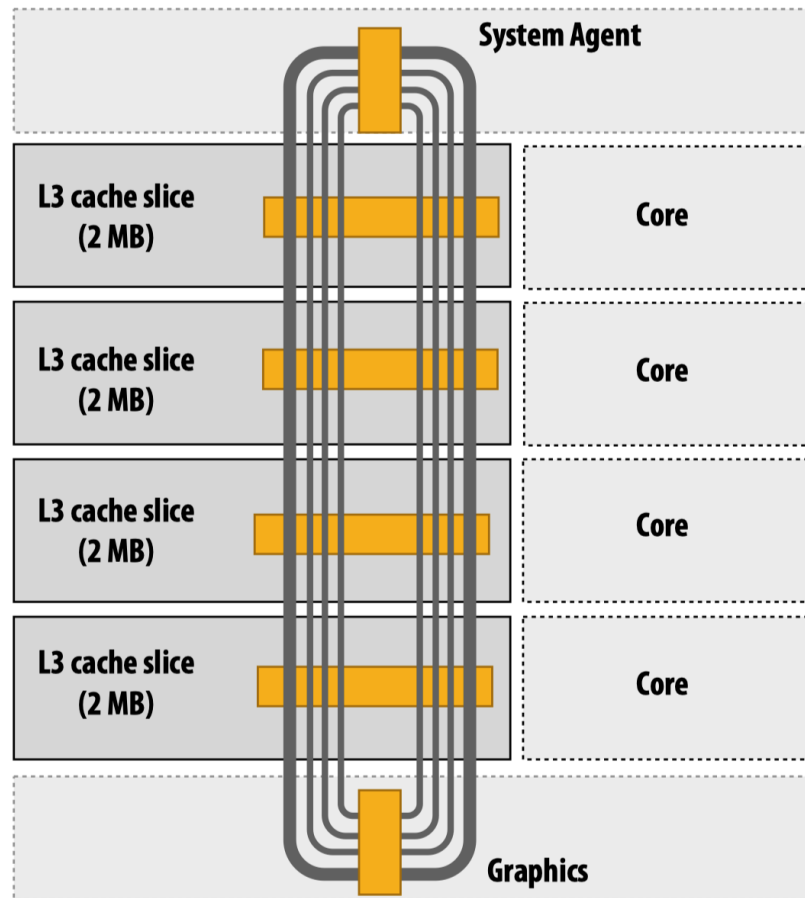
CMP服务器

■ 例1: Intel Core i7处理器



CMP服务器

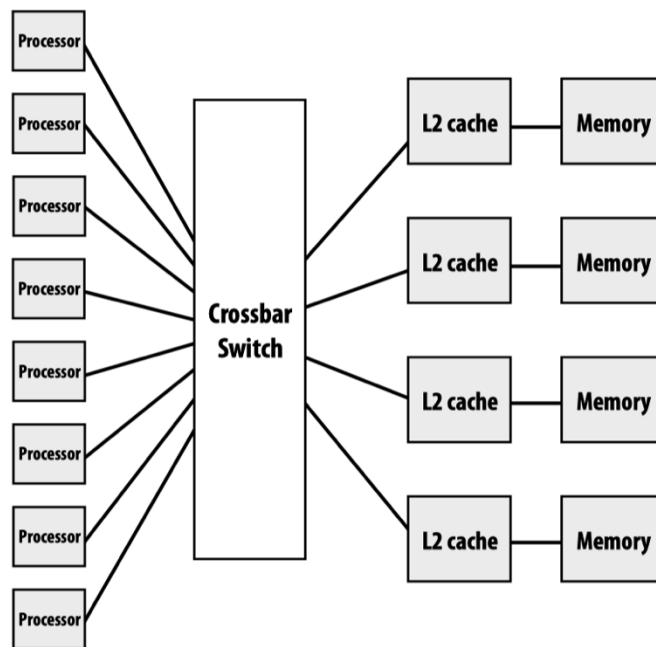
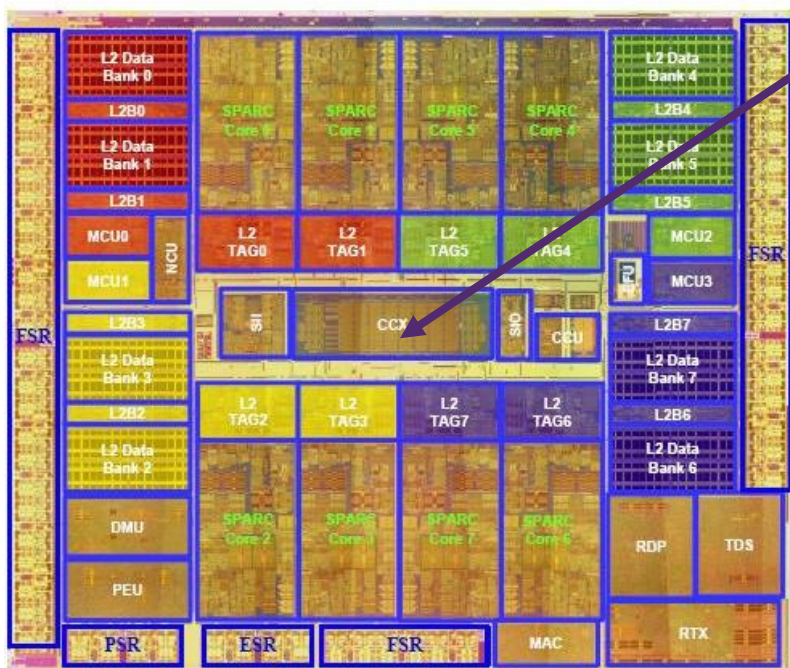
- Intel Core i7处理器内的互联方式
- 4个环
 - request
 - snoop
 - ack
 - data
- System Agent负责内存访问



CMP服务器

- 例2: SUN Niagara 2 (UltraSPARC T2)
 - 采用crossbar互联

Crossbar区域
约占一个核的大小



并行计算硬件模型 - 小结

- 共享内存：
 - 地址直接访问
 - UMA
 - NUMA
- 分布式内存：
 - 不可直接访问，需要显式通信、拷贝
 - 集群服务器（Cluster）
- 处理器视角：
 - SMP
 - CMP

并行编程模型

为什么并行编程模型

- 并行编程模型：
 - 对写并行程序很重要
 - 并行编程模型是并行计算机架构的抽象
 - 提供了在一个机器的物理实现和编程者可获得的这个机器的抽象概念之间的桥梁
 - 在硬件和软件工程师之间提供共同的理解层面
 - 换言之，并行程序员如何看待底层，逻辑视角
 - 编程模型：性能 $\leftrightarrow$ 易用性



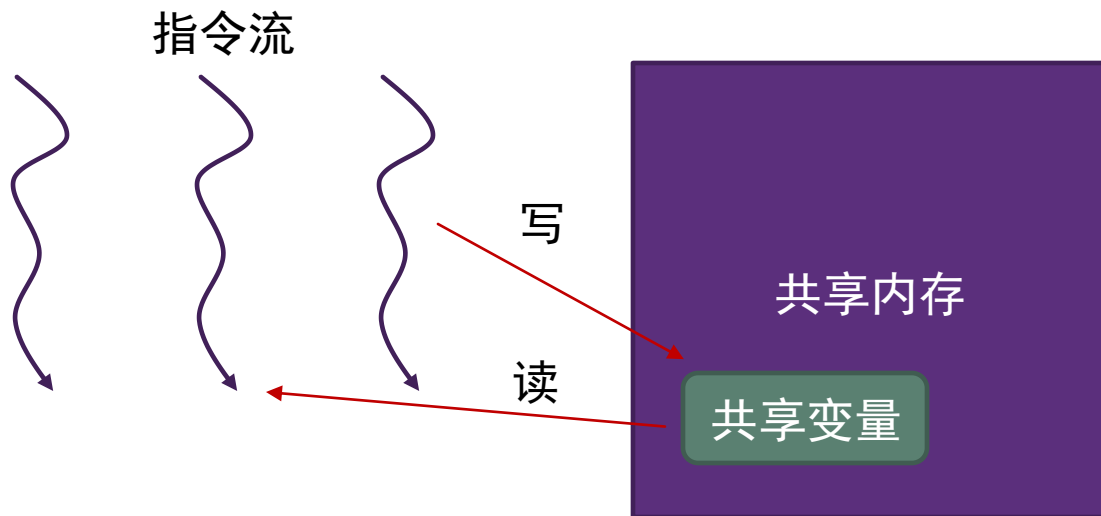
编程模型是应用与硬件之间的桥梁

并行编程模型

- 如何在众多硬件模型上编写并行程序？**软件抽象**
- 软件层次概念
 - **指令流**：规定操作并行、同步的方式
 - **数据**：规定数据的访问方式
- **典型并行编程模型：**
 - 共享内存模型
 - 消息传递模型
 - 数据并行模型

共享内存模型

- 指令流：不同指令流并发执行，指令流之间的同步采用**共享变量**的方式进行
- 数据：数据存放在**单一地址空间**，不同指令流均可通过地址**直接访问**数据
- 适用的硬件模型：**共享内存架构**



共享内存模型-1: pthread

- POSIX Thread (pthread)
- UNIX系统关于线程的一套标准接口
- C语言接口，通过函数库调用

主要接口	描述
pthread_create	创建线程
pthread_join	等待线程执行结束

共享内存模型-1: pthread

- 例子: Hello World
- 不同线程并发执行
- 共享内存:
 - 均可直接访问NUM_THREADS
 - 以及传入的thread_ids (同一数组的不同位置)

某次运行结果

```
[Main] Create thread 0
[Main] Create thread 1
[0/3] Hello
[1/3] Hello
[Main] Create thread 2
[Main] Join thread 0
[Main] Join thread 1
[Main] Join thread 2
[2/3] Hello
[Main] Exit
```

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

static const int NUM_THREADS = 3;
```

```
void* thread_func(void* data) {
    int tid = *(int*)data;
    printf("[%d/%d] Hello\n", tid, NUM_THREADS);

    return NULL;
}
```

线程主函数

```
int main() {
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];
```

```
// Create
for (int tid = 0; tid < NUM_THREADS; ++tid) {
    printf("[Main] Create thread %d\n", tid);
    thread_ids[tid] = tid;
    pthread_create(&threads[tid], NULL, thread_func, &thread_ids[tid]);
}
```

创建线程

```
// Join
for (int tid = 0; tid < NUM_THREADS; ++tid) {
    printf("[Main] Join thread %d\n", tid);
    pthread_join(threads[tid], NULL);
}
```

等待线程结束

```
printf("[Main] Exit\n");
return 0;
}
```

共享内存模型-2: OpenMP

- OpenMP是一套面向共享内存模型的并行程序设计标准
- OpenMP为C/C++, Fortran定义了一套编译指导语句规范, 由编译器提供支持
 - 目前主流编译器 (如gcc, clang, icc, pgi等) 均支持
 - 编译: gcc -fopenmp -o hello hello.c
- OpenMP底层基于Pthread实现

```
#include <omp.h>
#include <stdio.h>

static const int NUM_THREADS = 3;

int main() {
    #pragma omp parallel for num_threads(NUM_THREADS)
    for (int tid = 0; tid < NUM_THREADS; ++tid) {
        printf("[%d/%d] Hello\n", omp_get_thread_num(), omp_get_num_threads());
    }

    printf("[Main] Exit\n");
    return 0;
}
```

一行指导语句

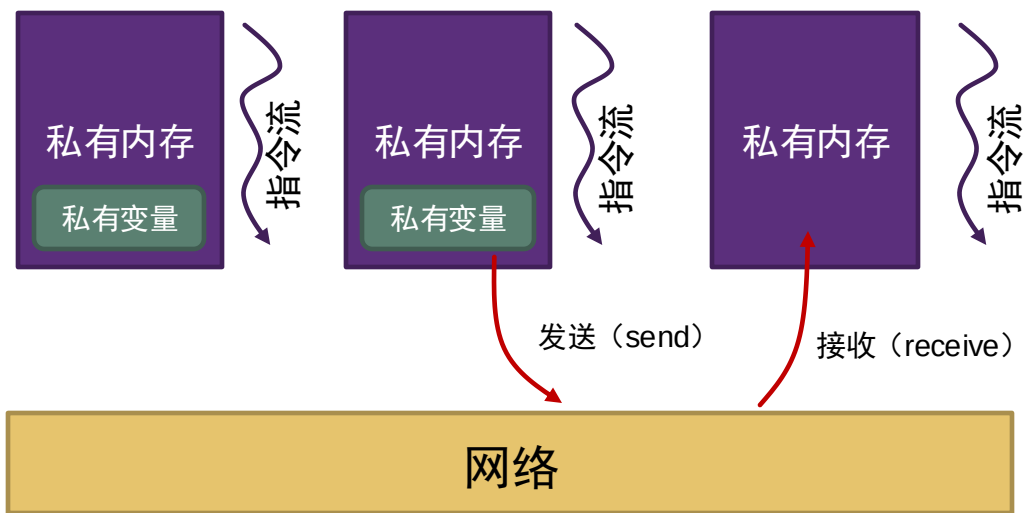
某次运行结果

```
[0/3] Hello
[1/3] Hello
[2/3] Hello
[Main] Exit
```

#pragma omp parallel for

消息传递模型

- **指令流**：不同指令流并发执行，指令流之间同步基于**通信**的方式进行
- **数据**：不同指令流拥有**独立的地址空间**，相互不可通过地址直接访问，需要**显式的通信**
 - 通信：通过消息的发送、接收操作来进行数据传输
- 适用的硬件模型：**分布式内存架构、共享内存架构**
 - 通信采用网络消息的方式实现



消息传递模型的实现-MPI

- Message Passing Interface (MPI) 是目前应用最为广泛的消息传递程序**事实标准**
- MPI有多种不同实现：
 - OpenMPI, Intel MPI, MPICH等
- 后面课程会详细介绍

常见接口	描述
MPI_Init	初始化MPI
MPI_Finalize	结束MPI
MPI_Comm_size	返回进程数
MPI_Comm_rank	返回当前进程ID
MPI_Send	一对一发送数据
MPI_Recv	一对一接收数据
MPI_Scatter	一对多发送数据
MPI_Gather	多对一接收数据

消息传递模型的实现-MPI示例

```
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(NULL, NULL);           初始化 MPI

    int comm_sz;
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);  获取总进程数

    int my_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);  获取当前进程编号

    int a[2];

    if (my_rank == 0) {
        int root_a[] = {0, 1, 2, 3, 4, 5};
        MPI_Scatter(root_a, 2, MPI_INT, a, 2, MPI_INT, 0, MPI_COMM_WORLD);
    } else {
        MPI_Scatter(NULL, 2, MPI_INT, a, 2, MPI_INT, 0, MPI_COMM_WORLD);
    }

    printf("[%d/%d] I get %d and %d.\n", my_rank, comm_sz, a[0], a[1]);

    MPI_Finalize();                 结束 MPI
    return 0;
}
```

某次运行结果

```
[0/3] I get 0 and 1.
[1/3] I get 2 and 3.
[2/3] I get 4 and 5.
```

- 编译: `mpicxx example.cpp -o example`
- 运行: `mpirun -n 3 ./example`

数据并行模型

- 在不同数据上并发执行相同逻辑
- **指令流**：逻辑上一个指令流
- **数据**：多份数据
- 适用的硬件模型：
 - SIMD
 - CPU中的向量处理单元
 - GPU
 - 分布式内存
 - MapReduce

数据并行模型: SIMD

■ Intel向量扩展指令

- <https://software.intel.com/sites/landingpage/IntrinsicsGuide>



Technologies

- ☐ MMX
- ☐ SSE
- ☐ SSE2
- ☐ SSE3
- ☐ SSSE3
- ☐ SSE4.1
- ☐ SSE4.2
- ☐ AVX
- ☐ AVX2
- ☐ FMA
- ☐ AVX-512
- ☐ KNC
- ☐ AMX
- ☐ SVM
- ☐ Other

Categories

- ☐ Application-Targeted
- ☐ Arithmetic
- ☐ Bit Manipulation
- ☐ Cast
- ☐ Compare
- ☐ Convert
- ☐ Cryptography
- ☐ Elementary Math
- Functions
 - ☐ General Support
 - ☐ Load
 - ☐ Logical
 - ☐ Mask
 - ☐ Miscellaneous
 - ☐ Memory

The Intel Intrinsics Guide is an interactive reference tool for Intel intrinsic instructions, which are C style functions that provide access to many Intel instructions - including Intel® SSE, AVX, AVX-512, and more - without the need to write assembly code.

_mm_search

```
void _mm_2intersect_epi32 (__m128i a, __m128i b, __mmask8* k1, __mmask8* k2)      vp2intersectd
void _mm256_2intersect_epi32 (__m256i a, __m256i b, __mmask8* k1, __mmask8* k2)  vp2intersectd
void _mm512_2intersect_epi32 (__m512i a, __m512i b, __mmask16* k1, __mmask16* k2) vp2intersectd
void _mm_2intersect_epi64 (__m128i a, __m128i b, __mmask8* k1, __mmask8* k2)      vp2intersectq
void _mm256_2intersect_epi64 (__m256i a, __m256i b, __mmask8* k1, __mmask8* k2)  vp2intersectq
void _mm512_2intersect_epi64 (__m512i a, __m512i b, __mmask8* k1, __mmask8* k2)  vp2intersectq
__m512i _mm512_4dpwssd_epi32 (__m512i src, __m512i a0, __m512i a1, __m512i a2, __m512i a3,
__m128i * b)      vp4dpwssd
__m512i _mm512_mask_4dpwssd_epi32 (__m512i src, __mmask16 k, __m512i a0, __m512i a1, __m512i a2,
__m512i a3, __m128i * b)      vp4dpwssd
__m512i _mm512_maskz_4dpwssd_epi32 (__mmask16 k, __m512i src, __m512i a0, __m512i a1, __m512i a2,
__m512i a3, __m128i * b)      vp4dpwssd
__m512i _mm512_4dpwssds_epi32 (__m512i src, __m512i a0, __m512i a1, __m512i a2, __m512i a3,
__m128i * b)      vp4dpwssds
__m512i _mm512_mask_4dpwssds_epi32 (__m512i src, __mmask16 k, __m512i a0, __m512i a1, __m512i
a2, __m512i a3, __m128i * b)      vp4dpwssds
__m512i _mm512_maskz_4dpwssds_epi32 (__mmask16 k, __m512i src, __m512i a0, __m512i a1, __m512i
a2, __m512i a3, __m128i * b)      vp4dpwssds
__m512 _mm512_4fmadd_ps (__m512 src, __m512 a0, __m512 a1, __m512 a2, __m512 a3, __m128 * b)      v4fmaddps
__m512 _mm512_mask_4fmadd_ps (__m512 src, __mmask16 k, __m512 a0, __m512 a1, __m512 a2, __m512
a3, __m128 * b)      v4fmaddps
__m512 _mm512_maskz_4fmadd_ps (__mmask16 k, __m512 src, __m512 a0, __m512 a1, __m512 a2, __m512
a3, __m128 * b)      v4fmaddps
__m128 _mm_4fmadd_ss (__m128 src, __m128 a0, __m128 a1, __m128 a2, __m128 a3, __m128 * b)      v4fmaddss
__m128 _mm_mask_4fmadd_ss (__m128 src, __mmask8 k, __m128 a0, __m128 a1, __m128 a2, __m128 a3,
__m128 * b)      v4fmaddss
```

数据并行模型: SIMD

Intel向量扩展指令举例

- 计算 $\sum_{i=0}^N A[i]$
- A 是32位整数类型（int）数组， N 为4的倍数

SSE Data Types (16 XMM Registers)

__m128	Float	Float	Float	Float	4x 32-bit float
__m128d	Double		Double		2x 64-bit double
__m128i	B	B	B	B	16x 8-bit byte
m128i	short	short	short	short	8x 16-bit short
__m128i	int	int	int	int	4x 32bit integer
__m128i	long long		long long		2x 64bit long
__m128i	doublequadword				1x 128-bit quad



Technologies

- ☐ MMX
- ☐ SSE
- ☒ SSE2
- ☐ SSE3
- ☐ SSSE3
- ☐ SSE4.1
- ☐ SSE4.2
- ☐ AVX
- ☐ AVX2
- ☐ FMA
- ☐ AVX-512
- ☐ KNC
- ☐ AMX
- ☐ SVMML
- ☐ Other

Categories

- ☐ Application-Targeted
 - ☐ Arithmetic
 - ☐ Bit Manipulation
 - ☐ Cast
 - ☐ Compare
 - ☐ Convert
 - ☐ Cryptography
 - ☐ Elementary Math
- Functions

The Intel Intrinsic Guide is an interactive reference tool for Intel in many Intel instructions - including Intel® SSE, AVX, AVX-512, and n

_mm_search

```
__m128i _mm_add_epi16 (__m128i a, __m128i b)
__m128i _mm_add_epi32 (__m128i a, __m128i b)
```

Synopsis

```
__m128i _mm_add_epi32 (__m128i a, __m128i b)
#include <emmintrin.h>
Instruction: padd xmm, xmm
CPUTID Flags: SSE2
```

Description

Add packed 32-bit integers in *a* and *b*, and store the results in *dst*.

Operation

```
FOR j := 0 to 3
  i := j*32
  dst[i+31:i] := a[i+31:i] + b[i+31:i]
ENDFOR
```

Performance

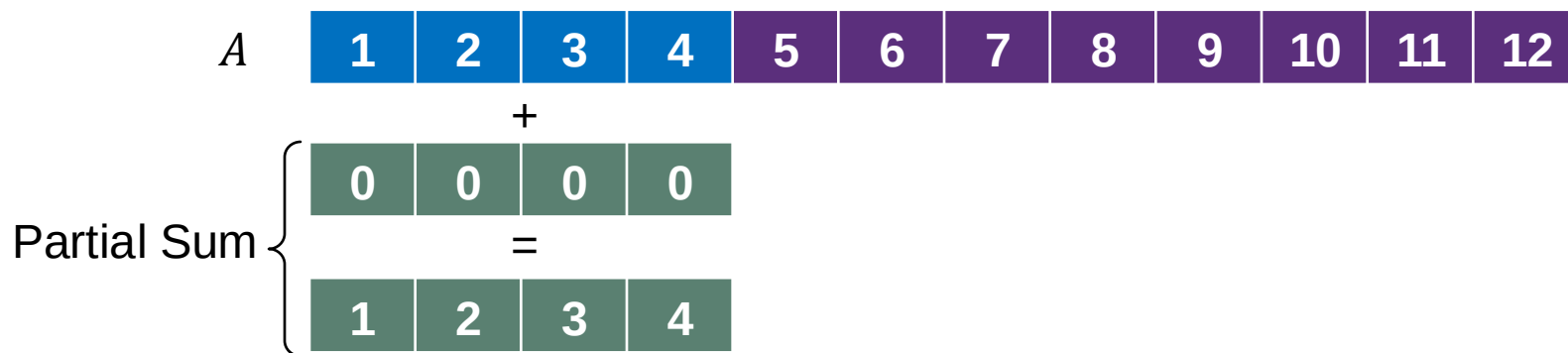
Architecture	Latency	Throughput (CPI)
Skylake	1	0.33
Broadwell	1	0.5
Haswell	1	0.5
Ivy Bridge	1	0.5

数据并行模型: SIMD

Intel向量扩展指令举例

- 计算 $\sum_{i=0}^N A[i]$
- A 是32位整数类型 (int) 数组
- N 为4的倍数

tmpA



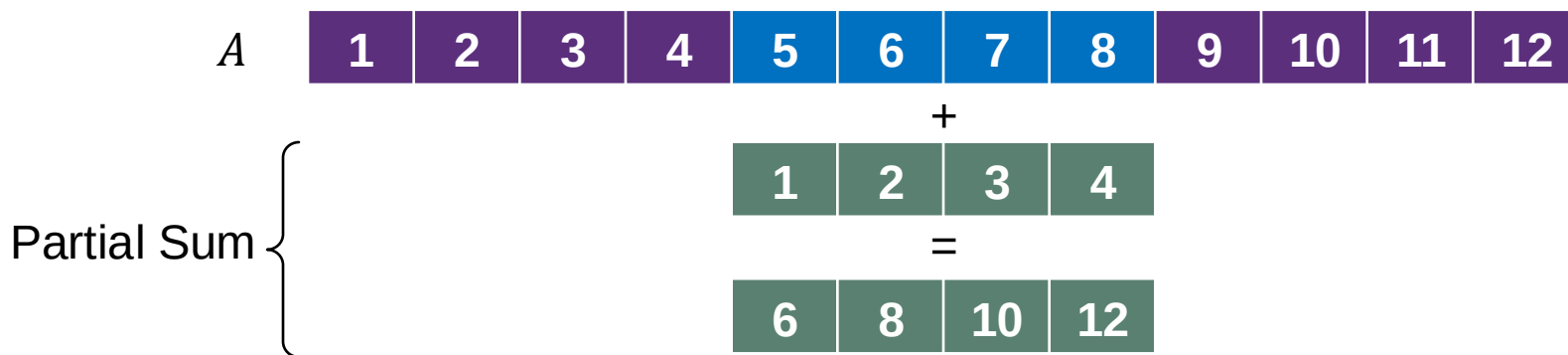
```
int add(int* A, int N) {  
    __m128i partial_sum = _mm_setzero_si128();  
    for (int i = 0; i < N; i += 4) {  
        __m128i tmpA = _mm_load_si128((const __m128i*)(A + i));  
        partial_sum = _mm_add_epi32(partial_sum, tmpA);  
    }  
  
    int outputs[4];  
    _mm_store_si128((__m128i*)outputs, partial_sum);  
  
    int sum = 0;  
    for (int i = 0; i < 4; ++i) sum += outputs[i];  
  
    return sum;  
}
```

数据并行模型: SIMD

Intel向量扩展指令举例

- 计算 $\sum_{i=0}^N A[i]$
- A 是32位整数类型 (int) 数组
- N 为4的倍数

tmpA



```
int add(int* A, int N) {  
    __m128i partial_sum = _mm_setzero_si128();  
    for (int i = 0; i < N; i += 4) {  
        __m128i tmpA = _mm_load_si128((const __m128i*)(A + i));  
        partial_sum = _mm_add_epi32(partial_sum, tmpA);  
    }  
  
    int outputs[4];  
    _mm_store_si128((__m128i*)outputs, partial_sum);  
  
    int sum = 0;  
    for (int i = 0; i < 4; ++i) sum += outputs[i];  
  
    return sum;  
}
```

数据并行模型: SIMD

Intel向量扩展指令举例

- 计算 $\sum_{i=0}^N A[i]$
- A 是32位整数类型 (int) 数组
- N 为4的倍数

tmpA

Partial Sum {

A	1	2	3	4	5	6	7	8	9	10	11	12
{									+			
									6	8	10	12
									=			
um									15	18	21	24

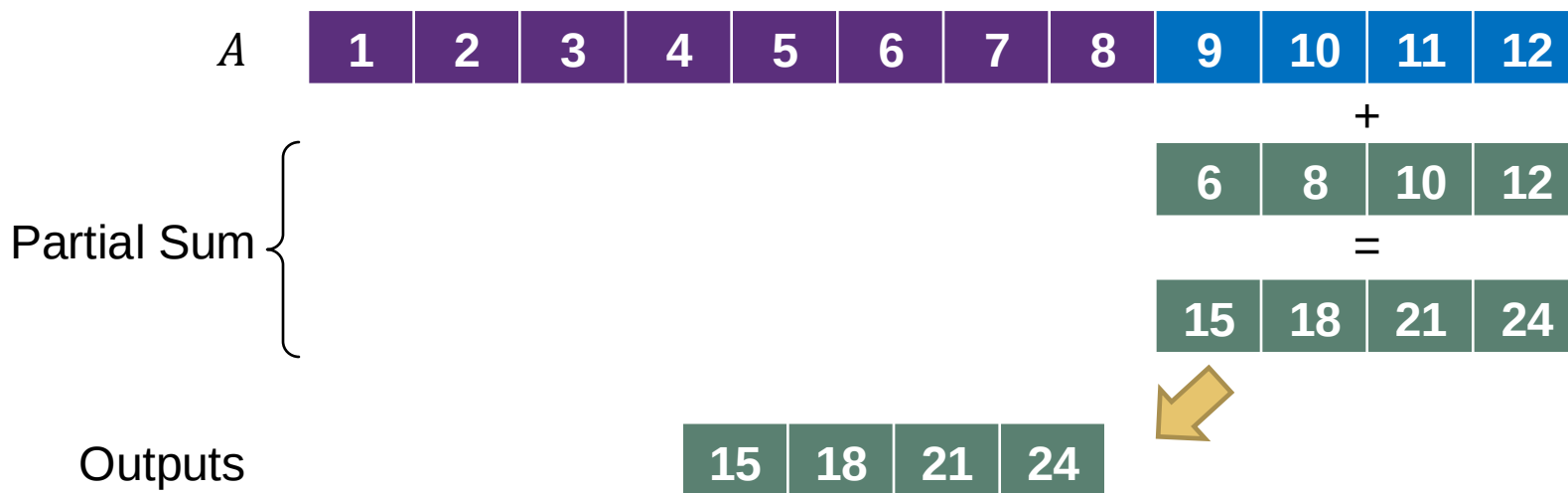
```
int add(int* A, int N) {  
    __m128i partial_sum = _mm_setzero_si128();  
    for (int i = 0; i < N; i += 4) {  
        __m128i tmpA = _mm_load_si128((const __m128i*)(A + i));  
        partial_sum = _mm_add_epi32(partial_sum, tmpA);  
    }  
  
    int outputs[4];  
    _mm_store_si128((__m128i*)outputs, partial_sum);  
  
    int sum = 0;  
    for (int i = 0; i < 4; ++i) sum += outputs[i];  
  
    return sum;  
}
```

数据并行模型: SIMD

Intel向量扩展指令举例

- 计算 $\sum_{i=0}^N A[i]$
- A 是32位整数类型 (int) 数组
- N 为4的倍数

tmpA



```
int add(int* A, int N) {  
    __m128i partial_sum = _mm_setzero_si128();  
    for (int i = 0; i < N; i += 4) {  
        __m128i tmpA = _mm_load_si128((const __m128i*)(A + i));  
        partial_sum = _mm_add_epi32(partial_sum, tmpA);  
    }  
  
    int outputs[4];  
    _mm_store_si128((__m128i*)outputs, partial_sum);  
  
    int sum = 0;  
    for (int i = 0; i < 4; ++i) sum += outputs[i];  
  
    return sum;  
}
```

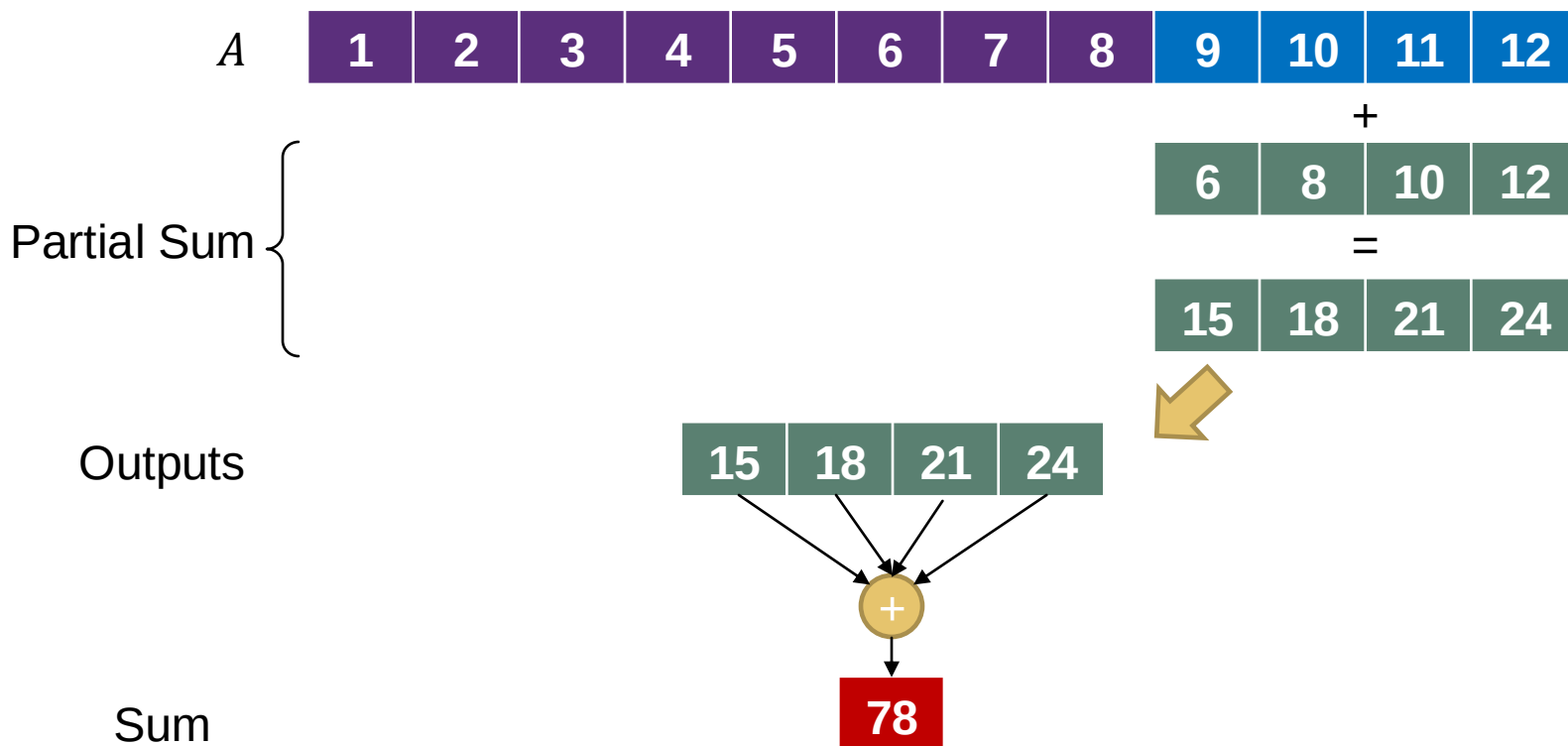
数据并行模型: SIMD

Intel向量扩展指令举例

- 计算 $\sum_{i=0}^N A[i]$
- A 是32位整数类型 (int) 数组
- N 为4的倍数

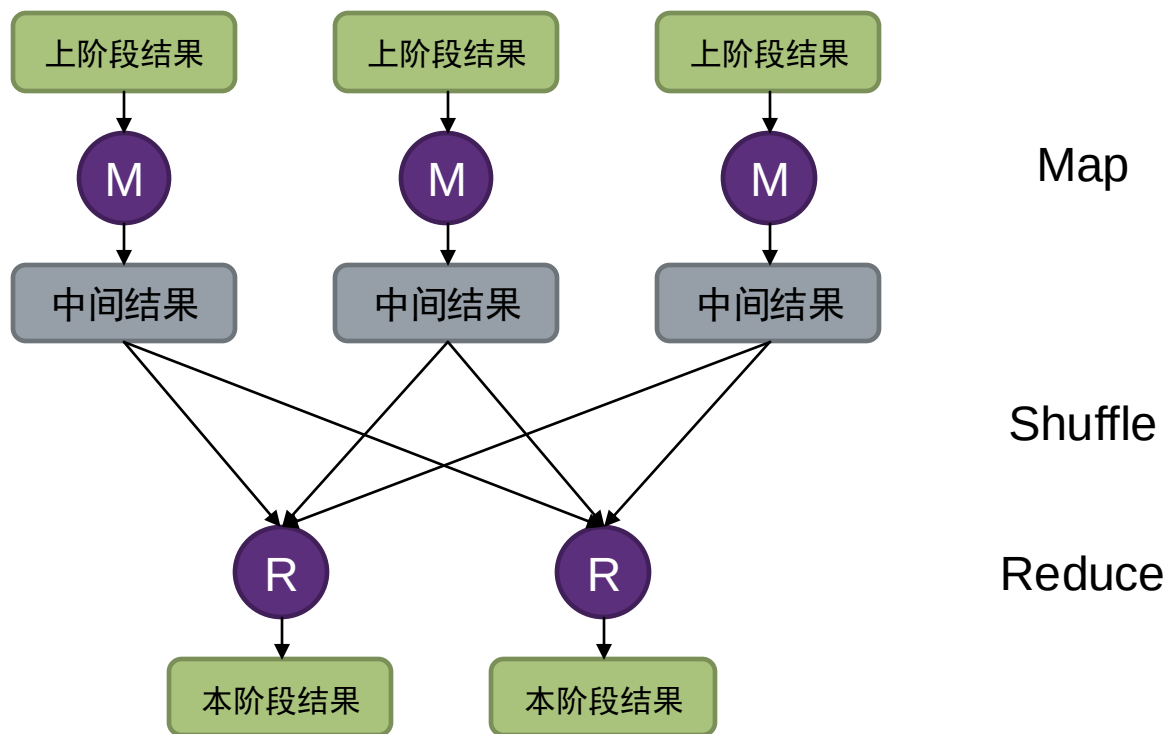
tmpA

```
int add(int* A, int N) {  
    __m128i partial_sum = _mm_setzero_si128();  
    for (int i = 0; i < N; i += 4) {  
        __m128i tmpA = _mm_load_si128((const __m128i*)(A + i));  
        partial_sum = _mm_add_epi32(partial_sum, tmpA);  
    }  
  
    int outputs[4];  
    _mm_store_si128((__m128i*)outputs, partial_sum);  
  
    int sum = 0;  
    for (int i = 0; i < 4; ++i) sum += outputs[i];  
  
    return sum;  
}
```



数据并行模型

- MapReduce编程模型：大数据重要编程模型
 - Map阶段：相同逻辑处理输入中的每个元素，生成中间结果
 - Reduce阶段：综合中间结果得到最终结果
- Map阶段适用数据并行模型

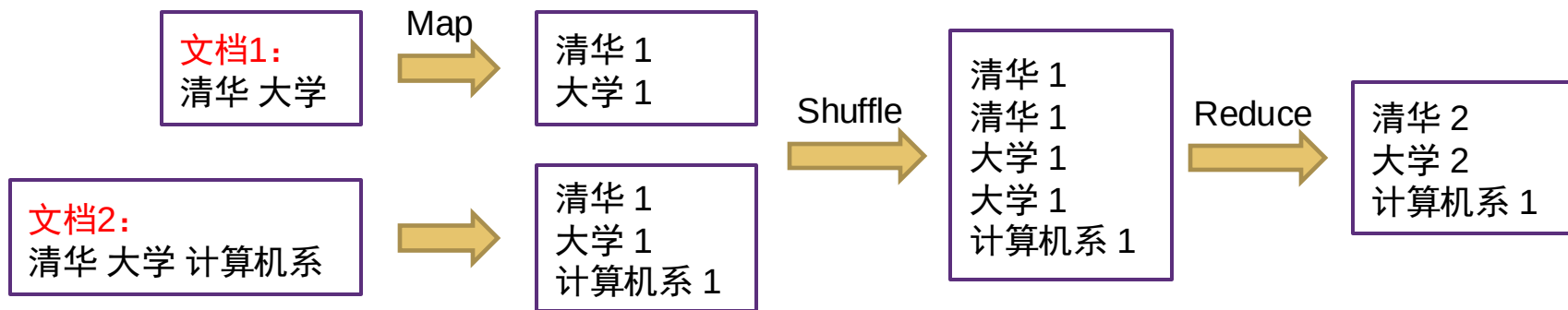
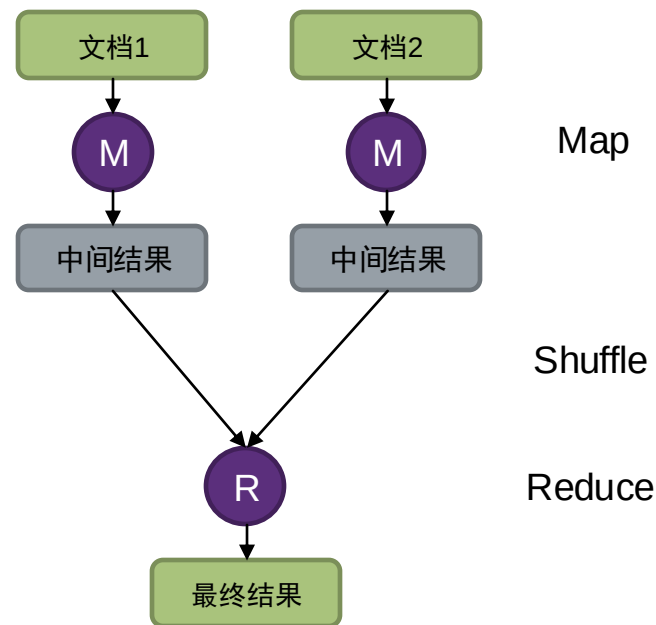


数据并行模型

MapReduce示例

单词统计：统计多个文档中出现单词的个数

```
function map(String name, String document):  
    // name: document name  
    // document: document contents  
    for each word w in document:  
        emit (w, 1)  
  
function reduce(String word, Iterator partialCounts):  
    // word: a word  
    // partialCounts: a list of partial counts  
    sum = 0  
    for each pc in partialCounts:  
        sum += pc  
    emit (word, sum)
```



并行编程模型-小结

■ 共享内存模型

- 指令流：并发执行，同步基于共享变量
- 数据：单一地址空间，地址直接访问

■ 消息传递模型

- 指令流：并发执行，同步基于通信
- 数据：独立地址空间，不可直接访问，需要显式通信

■ 数据并行模型

- 单一指令流并发作用于多个数据

	指令流	指令流同步	地址空间	数据访问
共享内存模型	多个并发	共享变量	单一地址空间	地址直接访问
消息传递模型	多个并发	通信	独立地址空间	显式通信
数据并行模型	单一	隐式	-	-

并行编程模型 vs. 硬件模型

- 并行编程模型和硬件模型并无固定对应关系，**但一般**：
 - 共享内存硬件 $\longleftrightarrow$ 共享内存模型
 - 分布式内存硬件 $\longleftrightarrow$ 消息传递模型
- **但是**，在共享内存硬件模型上实现消息传递模型也很普遍
 - 在单SMP系统上，通过进程间通信实现显示消息传递
- 在分布式内存硬件模型上实现共享内存模型也是可行的
 - 通过软件手段
 - 访问远程地址，触发Page Fault，底层通过通信手段把远程内存传到本地
 - 示例：
 - CUDA的Unified Memory
 - Distributed Global Address Space：在多节点的分布式内存上实现统一地址空间

混合多种并行编程模型

- 实际应用中往往混合使用多种并行模型
- 分布式深度学习训练
 - 多个节点采用消息传递模型
 - 例如：MPI AllReduce通信
 - 单节点内采用共享内存+消息传递+数据并行模型
 - 共享内存：多个IO线程与主计算线程
 - 消息传递：CPU内存数据拷贝到GPU内存中，GPU计算结果拷贝到CPU内存中
 - 数据并行：GPU计算
- 科学计算：超大规模集群，多核节点，异构计算设备

并行计算性能指标

加速比

■ **加速比**: $S(p) = \frac{T_S}{T_p}$

- T_S : 程序的串行版本执行时间
- T_p : 程序的并行版本执行时间, 使用的处理单元数目为 p

1. **线性加速**: $S(p) = p$

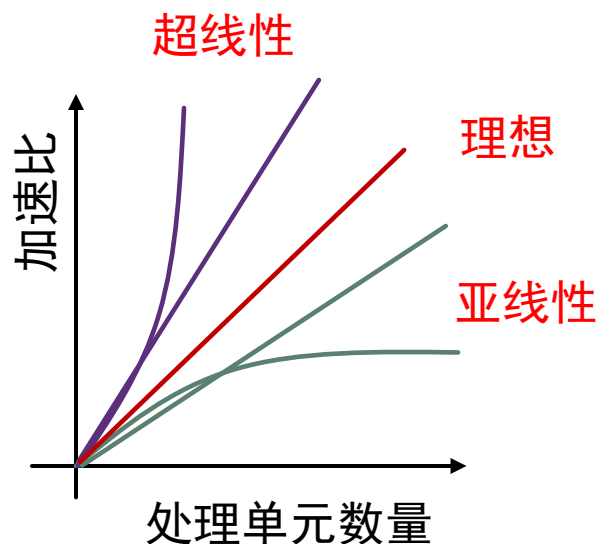
- 理论中的理想最大加速比

2. **超线性加速**: $S(p) > p$

- 实际中有时发生
 - 例如: 数据缓存在cache

3. **亚线性加速**:

- 典型加速情况



理想线性加速

- 难以实现理想线性加速的原因
 - 程序中不是所有部分都可以并行
 - 导致处理器闲置
 - 并行处理引入通信和同步开销
 - 通常是主要因素
 - 想办法避免
 - 并行处理可能引入额外计算
 - 为了减轻通信和同步开销，可能会做少量重复计算

总结

- 并行计算概念
 - Flynn分类法
- 并行计算硬件模型
 - 共享内存
 - 分布式内存
- 并行编程模型
 - 共享内存模型
 - 消息传递模型
 - 数据并行模型
- 并行计算性能指标
 - 加速比

作业-1

- 使用 MPI 和 OpenMP 并行下述程序，报告加速比

```
void pow_a(int *a, int *b, int n, int m) {  
    for (int i = 0; i < n; i++) {  
        int x = 1;  
        for (int j = 0; j < m; j++)  
            x *= a[i];  
        b[i] = x;  
    }  
}
```

- 目的：熟悉环境配置、集群使用、各种工具等
- 详见网络学堂

课后阅读

- 阅读《高性能计算之并行编程技术-MPI并行程序设计》1-5章
 - 电子版见网络学堂
- 阅读文档：
<https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>